

PRACTICE WORKBOOK

300 Python Programs with Solutions

Problems · Solutions · Sample Outputs ·
Concepts

From beginner fundamentals to interview-ready practice

Created By : Kartik Mungole · @kartikmungole

CONTENTS

Table of Contents

16 sections, 300 problems — arranged from basics to interview classics

01	Basics, Input/Output & Variables	p. 3
02	Operators & Math	p. 13
03	Conditionals (if / elif / else)	p. 33
04	Loops (for / while)	p. 48
05	Star & Number Patterns	p. 73
06	Strings	p. 88
07	Lists	p. 113
08	Tuples & Sets	p. 138
09	Dictionaries	p. 150
10	Functions, Lambda & Recursion	p. 170
11	Comprehensions & Pythonic Idioms	p. 195
12	Object-Oriented Programming	p. 207
13	File Handling	p. 229
14	Exception Handling & Standard Modules	p. 241
15	Searching, Sorting & DSA Classics	p. 256
16	Real-World Mini Programs & Interview Classics	p. 281

SECTION 1

Basics, Input/Output & Variables

(Problems 1–15)

Problem 1. Hello World

Write a program that prints the message "Hello, World!" to the screen.

SOLUTION

```
print("Hello, World!")
```

OUTPUT

```
Hello, World!
```

Concept: the `print()` function and string literals.

Problem 2. Print Name and Age from Input

Ask the user for their name and age, then print both values on separate lines.

SOLUTION

```
name = input("Enter your name: ")
age = input("Enter your age: ")

print("Name:", name)
print("Age:", age)
```

OUTPUT

```
Enter your name: Riya
Enter your age: 21
Name: Riya
Age: 21
```

Concept: `input()` always returns a string; `print()` with multiple arguments.

Problem 3. Sum of Two Numbers

Read two integers from the user and print their sum.

SOLUTION

```
a = int(input("Enter first number: "))    # int() converts text to a whole
number
b = int(input("Enter second number: "))

print("Sum =", a + b)
```

OUTPUT

```
Enter first number: 12
Enter second number: 8
Sum = 20
```

Concept: converting input strings to int before doing arithmetic.

Problem 4. Swap Two Numbers Using a Temp Variable

Given two variables $a = 5$ and $b = 10$, swap their values using a third (temporary) variable and print the result.

SOLUTION

```
a = 5
b = 10
print("Before swap: a =", a, ", b =", b)

temp = a    # keep a's value safe
a = b
b = temp

print("After swap: a =", a, ", b =", b)
```

OUTPUT

```
Before swap: a = 5 , b = 10
After swap: a = 10 , b = 5
```

Concept: variable assignment and using a temporary variable to hold a value.

Problem 5. Swap Two Numbers Without a Temp Variable

Swap the values of two variables without using any extra variable.

SOLUTION

```
a = 5
b = 10
print("Before swap: a =", a, ", b =", b)

a, b = b, a    # Python swaps both values in one step

print("After swap: a =", a, ", b =", b)
```

OUTPUT

```
Before swap: a = 5 , b = 10
After swap:  a = 10 , b = 5
```

Concept: tuple unpacking (multiple assignment) for swapping values.

Problem 6. Type Conversion

Start with the string "25". Convert it to an int and a float, then convert the number 99 back to a string. Print each value with its type.

SOLUTION

```
num_str = "25"

num_int = int(num_str)      # string to integer
num_float = float(num_str) # string to float
back_to_str = str(99)      # number to string

print(num_int, type(num_int))
print(num_float, type(num_float))
print(back_to_str, type(back_to_str))
```

OUTPUT

```
25 <class 'int'>
25.0 <class 'float'>
99 <class 'str'>
```

Concept: type casting with int(), float() and str().

Problem 7. type() of Different Values

Print the data type of these values: 10, 3.14, "hello", True, and [1, 2, 3].

SOLUTION

```
print(type(10))
print(type(3.14))
print(type("hello"))
print(type(True))
print(type([1, 2, 3]))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>
```

Concept: built-in data types and the type() function.

Problem 8. f-string Formatting

Given a name and a marks value with many decimals, print a sentence using an f-string that shows the marks rounded to 2 decimal places.

SOLUTION

```
name = "Aman"
marks = 87.5678

# {marks:.2f} means: show the number with exactly 2 digits after the decimal
point
print(f"{name} scored {marks:.2f} marks.")
```

OUTPUT

```
Aman scored 87.57 marks.
```

Concept: f-strings and the .2f format specifier for rounding display.

Problem 9. Multiple Inputs on One Line

Read two numbers typed on a single line (separated by a space) and print their product.

SOLUTION

```
x, y = input("Enter two numbers: ").split() # split() breaks the line at
spaces
x = int(x)
y = int(y)

print("Product =", x * y)
```

OUTPUT

```
Enter two numbers: 6 7
Product = 42
```

Concept: splitting one input line into multiple values with `split()`.

Problem 10. Area of a Rectangle

Read the length and width of a rectangle from the user and print its area.

SOLUTION

```
length = float(input("Enter length: "))
width = float(input("Enter width: "))

area = length * width
print("Area of rectangle =", area)
```

OUTPUT

```
Enter length: 8.5
Enter width: 4
Area of rectangle = 34.0
```

Concept: float input and a simple arithmetic formula.

Problem 11. Average of Three Numbers

Read three integers entered on one line and print their average.

SOLUTION

```
# map() applies int() to every piece produced by split()
a, b, c = map(int, input("Enter three numbers: ").split())

average = (a + b + c) / 3
print(f"Average = {average:.2f}")
```

OUTPUT

```
Enter three numbers: 10 20 25
Average = 18.33
```

Concept: `map()` with `split()` to convert several inputs at once.

Section 1 · Basics, Input/Output & Variables (continued)

Problem 12. Convert Seconds to Hours, Minutes, Seconds

Read a total number of seconds and convert it into hours, minutes and seconds.

SOLUTION

```
total = int(input("Enter seconds: "))

hours = total // 3600          # whole hours
minutes = (total % 3600) // 60 # leftover minutes
seconds = total % 60          # leftover seconds

print(f"{hours} hour(s) {minutes} minute(s) {seconds} second(s)")
```

OUTPUT

```
Enter seconds: 3672
1 hour(s) 1 minute(s) 12 second(s)
```

Concept: floor division (//) and modulo (%) for unit conversion.

Section 1 · Basics, Input/Output & Variables (continued)

Problem 13. ASCII Value of a Character

Read one character and print its ASCII value. Also show which character has ASCII value 65.

SOLUTION

```
ch = input("Enter a character: ")

print("ASCII value of", ch, "is", ord(ch)) # ord():
character to number
print("Character with ASCII 65 is", chr(65)) # chr():
number to character
```

OUTPUT

```
Enter a character: a
ASCII value of a is 97
Character with ASCII 65 is A
```

Concept: `ord()` and `chr()` for character–ASCII conversion.

Section 1 · Basics, Input/Output & Variables (continued)

Problem 14. Kilometers to Miles

Read a distance in kilometers and convert it to miles (1 km = 0.621371 miles). Show the answer rounded to 2 decimal places.

SOLUTION

```
km = float(input("Enter distance in km: "))

miles = km * 0.621371
print(f"{km} km = {miles:.2f} miles")
```

OUTPUT

```
Enter distance in km: 5
5.0 km = 3.11 miles
```

Concept: unit conversion with a constant and f-string rounding.

Section 1 · Basics, Input/Output & Variables (continued)

Problem 15. String Repetition and Concatenation

Read a word and a number n. Print the word joined with itself using +, then print the word repeated n times using *.

SOLUTION

```
word = input("Enter a word: ")
n = int(input("Enter repeat count: "))

joined = word + word    # + joins (concatenates) strings
repeated = word * n     # * repeats a string

print("Joined:", joined)
print("Repeated:", repeated)
```

OUTPUT

```
Enter a word: ha
Enter repeat count: 4
Joined: haha
Repeated: hahahaha
```

Concept: string concatenation (+) and repetition (*) operators.

SECTION 2

Operators & Math

(Problems 16–35)

Problem 16. Even or Odd

Read an integer and print whether it is even or odd.

SOLUTION

```
num = int(input("Enter a number: "))

if num % 2 == 0:    # a number is even if it leaves no
    remainder when divided by 2
    print(num, "is Even")
else:
    print(num, "is Odd")
```

OUTPUT

```
Enter a number: 7
7 is Odd
```

Concept: the modulo operator (%) to test divisibility.

Section 2 · Operators & Math (continued)

Problem 17. Floor Division and Modulo Demonstration

For the numbers 17 and 5, show the results of `/`, `//`, and `%`. Also show `//` and `%` with -17 to see how Python handles negatives.

SOLUTION

```
a = 17
b = 5

print("17 / 5    =", a / b)    # true division, gives a
float
print("17 // 5   =", a // b)   # floor division, rounds
DOWN
print("17 % 5    =", a % b)    # remainder
print("-17 // 5  =", -a // b)   # floor of -3.4 is -4, not
-3
print("-17 % 5   =", -a % b)   # remainder always matches
the divisor's sign
```

OUTPUT

```
17 / 5    = 3.4
17 // 5    = 3
17 % 5     = 2
-17 // 5   = -4
-17 % 5    = 3
```

Concept: difference between `/`, `//` and `%`, including negative numbers.

Section 2 · Operators & Math (continued)

Problem 18. Area of a Circle

Read the radius of a circle and print its area using the formula $\text{area} = \pi r^2$.

SOLUTION

```
import math

radius = float(input("Enter radius: "))
area = math.pi * radius ** 2    # ** means "raised to the
                                # power of"

print(f"Area of circle = {area:.2f}")
```

OUTPUT

```
Enter radius: 7
Area of circle = 153.94
```

Concept: importing the math module and the exponent operator (**).

Section 2 · Operators & Math (continued)

Problem 19. Celsius to Fahrenheit

Read a temperature in Celsius and convert it to Fahrenheit using $F = C \times 9/5 + 32$.

SOLUTION

```
celsius = float(input("Enter temperature in Celsius: "))  
  
fahrenheit = celsius * 9 / 5 + 32  
print(f"{celsius}°C = {fahrenheit}°F")
```

OUTPUT

```
Enter temperature in Celsius: 37  
37.0°C = 98.6°F
```

Concept: applying a mathematical formula with operator precedence.

Section 2 · Operators & Math (continued)

Problem 20. Fahrenheit to Celsius

Read a temperature in Fahrenheit and convert it to Celsius using $C = (F - 32) \times 5/9$.

SOLUTION

```
fahrenheit = float(input("Enter temperature in Fahrenheit:
"))

celsius = (fahrenheit - 32) * 5 / 9
print(f"{fahrenheit}°F = {celsius:.1f}°C")
```

OUTPUT

```
Enter temperature in Fahrenheit: 98.6
98.6°F = 37.0°C
```

Concept: parentheses to control the order of operations.

Section 2 · Operators & Math (continued)

Problem 21. Simple Interest

Read principal, rate (% per year) and time (years), then compute simple interest using $SI = P \times R \times T / 100$.

SOLUTION

```
p = float(input("Enter principal: "))
r = float(input("Enter rate (% per year): "))
t = float(input("Enter time (years): "))

si = p * r * t / 100
print("Simple Interest =", si)
```

OUTPUT

```
Enter principal: 10000
Enter rate (% per year): 5
Enter time (years): 2
Simple Interest = 1000.0
```

Concept: translating a real-world formula into code.

Section 2 · Operators & Math (continued)

Problem 22. Compound Interest

Read principal, rate and time, then compute the final amount $A = P(1 + R/100)^T$ and the compound interest $CI = A - P$.

SOLUTION

```
p = float(input("Enter principal: "))
r = float(input("Enter rate (% per year): "))
t = float(input("Enter time (years): "))

amount = p * (1 + r / 100) ** t
ci = amount - p

print(f"Final amount = {amount:.2f}")
print(f"Compound interest = {ci:.2f}")
```

OUTPUT

```
Enter principal: 10000
Enter rate (% per year): 5
Enter time (years): 2
Final amount = 11025.00
Compound interest = 1025.00
```

Concept: exponentiation (**) inside a financial formula.

Section 2 · Operators & Math (continued)

Problem 23. Sum of Digits

Read an integer and print the sum of its digits. For example, 4578 gives $4 + 5 + 7 + 8 = 24$.

SOLUTION

```
num = int(input("Enter a number: "))

total = 0
n = num
while n > 0:
    total += n % 10    # n % 10 gives the last digit
    n //= 10          # n //= 10 removes the last digit

print("Sum of digits of", num, "is", total)
```

OUTPUT

```
Enter a number: 4578
Sum of digits of 4578 is 24
```

Concept: extracting digits with `% 10` and `// 10` in a while loop.

Section 2 · Operators & Math (continued)

Problem 24. Reverse a Number

Read an integer and print it with its digits reversed. For example, 1234 becomes 4321.

SOLUTION

```
num = int(input("Enter a number: "))

rev = 0
n = num
while n > 0:
    rev = rev * 10 + n % 10    # shift rev left, then
    attach the last digit of n
    n //= 10

print("Reverse of", num, "is", rev)
```

OUTPUT

```
Enter a number: 1234
Reverse of 1234 is 4321
```

Concept: building a reversed number digit by digit.

Section 2 · Operators & Math (continued)

Problem 25. Count Digits in a Number

Read an integer and count how many digits it has, without converting it to a string.

SOLUTION

```
num = int(input("Enter a number: "))

count = 0
n = num
while n > 0:
    count += 1
    n //= 10

print(num, "has", count, "digits")
```

OUTPUT

```
Enter a number: 45678
45678 has 5 digits
```

Concept: counting loop iterations while stripping digits.

Section 2 · Operators & Math (continued)

Problem 26. Power Without Using **

Read a base and an exponent, and compute base raised to the exponent using a loop instead of the ** operator.

SOLUTION

```
base = int(input("Enter base: "))
exp = int(input("Enter exponent: "))

result = 1
for _ in range(exp):    # multiply by base, exp times
    result *= base

print(f"{base} to the power {exp} = {result}")
```

OUTPUT

```
Enter base: 2
Enter exponent: 10
2 to the power 10 = 1024
```

Concept: repeated multiplication with a for loop and an accumulator variable.

Section 2 · Operators & Math (continued)

Problem 27. Palindrome Number

Read a number and check whether it reads the same backwards (e.g., 121 is a palindrome, 123 is not).

SOLUTION

```
num = int(input("Enter a number: "))

rev = 0
n = num
while n > 0:
    rev = rev * 10 + n % 10
    n //= 10

if rev == num:
    print(num, "is a palindrome")
else:
    print(num, "is not a palindrome")
```

OUTPUT

```
Enter a number: 121
121 is a palindrome
```

Concept: reversing a number and comparing it with the original.

Section 2 · Operators & Math (continued)

Problem 28. Factorial of a Number

Read a non-negative integer n and print $n!$ (the product $1 \times 2 \times \dots \times n$). Note that $0! = 1$.

SOLUTION

```
n = int(input("Enter a number: "))

factorial = 1
for i in range(1, n + 1):
    factorial *= i

print(f"{n}! = {factorial}")
```

OUTPUT

```
Enter a number: 5
5! = 120
```

Concept: running product using `range(1, n + 1)`.

Section 2 · Operators & Math (continued)

Problem 29. All Factors of a Number

Read a number and print all of its factors (numbers that divide it exactly).

SOLUTION

```
num = int(input("Enter a number: "))

print("Factors of", num, "are:")
for i in range(1, num + 1):
    if num % i == 0:           # i is a factor if it
        divides num exactly
        print(i, end=" ")     # end=" " prints on the same
line
print()
```

OUTPUT

```
Enter a number: 36
Factors of 36 are:
1 2 3 4 6 9 12 18 36
```

Concept: testing divisibility for every value in a range.

Section 2 · Operators & Math (continued)

Problem 30. Prime Number Check

Read a number and check whether it is prime. A prime number has exactly two factors: 1 and itself.

SOLUTION

```
num = int(input("Enter a number: "))

is_prime = num > 1
# only need to test divisors up to the square root of num
for i in range(2, int(num ** 0.5) + 1):
    if num % i == 0:
        is_prime = False
        break    # one divisor is enough to decide

if is_prime:
    print(num, "is a prime number")
else:
    print(num, "is not a prime number")
```

OUTPUT

```
Enter a number: 29
29 is a prime number
```

Concept: square-root optimization, boolean flag and break.

Section 2 · Operators & Math (continued)

Problem 31. Armstrong Number

Check whether a number equals the sum of its own digits, each raised to the power of the digit count. Example: $153 = 1^3 + 5^3 + 3^3$.

SOLUTION

```
num = int(input("Enter a number: "))

digits = len(str(num))    # how many digits the number has
total = 0
n = num
while n > 0:
    total += (n % 10) ** digits
    n //= 10

if total == num:
    print(num, "is an Armstrong number")
else:
    print(num, "is not an Armstrong number")
```

OUTPUT

```
Enter a number: 153
153 is an Armstrong number
```

Concept: digit extraction combined with powers and a running sum.

Section 2 · Operators & Math (continued)

Problem 32. Perfect Number Check

A perfect number equals the sum of its factors excluding itself (e.g., $28 = 1 + 2 + 4 + 7 + 14$). Check whether a given number is perfect.

SOLUTION

```
num = int(input("Enter a number: "))

total = 0
for i in range(1, num):      # stop before num itself
    if num % i == 0:
        total += i

if total == num:
    print(num, "is a perfect number")
else:
    print(num, "is not a perfect number")
```

OUTPUT

```
Enter a number: 28
28 is a perfect number
```

Concept: summing proper divisors with a loop and comparison.

Section 2 · Operators & Math (continued)

Problem 33. GCD of Two Numbers

Read two numbers and find their greatest common divisor (GCD) using the Euclidean algorithm.

SOLUTION

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

x, y = a, b
while y != 0:
    x, y = y, x % y    # replace (x, y) with (y, remainder)
    until remainder is 0

print(f"GCD of {a} and {b} is {x}")
```

OUTPUT

```
Enter first number: 48
Enter second number: 36
GCD of 48 and 36 is 12
```

Concept: the Euclidean algorithm using repeated modulo.

Section 2 · Operators & Math (continued)

Problem 34. LCM of Two Numbers

Read two numbers and find their least common multiple (LCM) using the relation $\text{LCM}(a, b) = a \times b / \text{GCD}(a, b)$.

SOLUTION

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

x, y = a, b
while y != 0:          # first find the GCD
    x, y = y, x % y

lcm = a * b // x       # // keeps the result an integer
print(f"LCM of {a} and {b} is {lcm}")
```

OUTPUT

```
Enter first number: 4
Enter second number: 6
LCM of 4 and 6 is 12
```

Concept: deriving LCM from GCD using integer division.

Section 2 · Operators & Math (continued)

Problem 35. Roots of a Quadratic Equation

Read coefficients a , b , c of $ax^2 + bx + c = 0$ and print its roots using the quadratic formula. Handle real-and-distinct, real-and-equal, and complex cases.

SOLUTION

```
import math

a = float(input("Enter a: "))
b = float(input("Enter b: "))
c = float(input("Enter c: "))

d = b ** 2 - 4 * a * c    # discriminant decides the type of roots

if d > 0:
    root1 = (-b + math.sqrt(d)) / (2 * a)
    root2 = (-b - math.sqrt(d)) / (2 * a)
    print(f"Two real roots: {root1} and {root2}")
elif d == 0:
    root = -b / (2 * a)
    print(f"One repeated root: {root}")
else:
    real = -b / (2 * a)
    imag = math.sqrt(-d) / (2 * a)
    print(f"Complex roots: {real} + {imag}i and {real} - {imag}i")
```

OUTPUT

```
Enter a: 1
Enter b: -5
Enter c: 6
Two real roots: 3.0 and 2.0
```

Concept: the discriminant, `math.sqrt()` and branching on three cases.

SECTION 3

Conditionals (if / elif / else)

(Problems 36–50)

Problem 36. Positive, Negative or Zero

Read a number and print whether it is positive, negative or zero.

SOLUTION

```
num = float(input("Enter a number: "))

if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")
```

OUTPUT

```
Enter a number: -8
Negative
```

Concept: the basic if / elif / else ladder.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 37. Vowel or Consonant

Read a single alphabet character and print whether it is a vowel or a consonant.

SOLUTION

```
ch = input("Enter a letter: ")

if ch.lower() in "aeiou":    # lower() makes the check
    print(ch, "is a Vowel")  # case-insensitive
else:
    print(ch, "is a Consonant")
```

OUTPUT

```
Enter a letter: E
E is a Vowel
```

Concept: the `in` operator for membership testing and `lower()` for case handling.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 38. Voting Eligibility

Read a person's age and print whether they are eligible to vote (age 18 or above).

SOLUTION

```
age = int(input("Enter your age: "))

if age ≥ 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote yet.")
    print("You can vote after", 18 - age, "year(s).")
```

OUTPUT

```
Enter your age: 16
You are not eligible to vote yet.
You can vote after 2 year(s).
```

Concept: comparison operators inside an if / else decision.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 39. Largest of Three Numbers

Read three numbers and print the largest one using if / elif / else (without the built-in max()).

SOLUTION

```
a, b, c = map(int, input("Enter three numbers: ").split())

if a ≥ b and a ≥ c:
    largest = a
elif b ≥ a and b ≥ c:
    largest = b
else:
    largest = c

print("Largest number is", largest)
```

OUTPUT

```
Enter three numbers: 12 45 7
Largest number is 45
```

Concept: combining comparisons with the and operator.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 40. Leap Year Check

Read a year and check if it is a leap year. A year is a leap year if it is divisible by 4 but not by 100, or if it is divisible by 400.

SOLUTION

```
year = int(input("Enter a year: "))

if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print(year, "is a leap year")
else:
    print(year, "is not a leap year")
```

OUTPUT

```
Enter a year: 2024
2024 is a leap year
```

Concept: combining and / or with parentheses to express a rule.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 41. Digit, Alphabet or Special Character

Read a single character and print whether it is a digit, an alphabet letter, or a special character.

SOLUTION

```
ch = input("Enter a character: ")

if ch.isdigit():
    print(ch, "is a Digit")
elif ch.isalpha():
    print(ch, "is an Alphabet letter")
else:
    print(ch, "is a Special character")
```

OUTPUT

```
Enter a character: @
@ is a Special character
```

Concept: string helper methods `isdigit()` and `isalpha()` in a decision ladder.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 42. Grade Calculator

Read marks out of 100 and print the grade: A for 90+, B for 80–89, C for 70–79, D for 60–69, and F below 60.

SOLUTION

```
marks = int(input("Enter marks (0-100): "))

if marks ≥ 90:
    grade = "A"
elif marks ≥ 80:
    grade = "B"
elif marks ≥ 70:
    grade = "C"
elif marks ≥ 60:
    grade = "D"
else:
    grade = "F"

print("Grade:", grade)
```

OUTPUT

```
Enter marks (0-100): 84
Grade: B
```

Concept: an elif ladder where the first true condition wins.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 43. Age-Based Ticket Price

A theme park charges: under 5 years free, 5–17 years Rs. 100, 18–59 years Rs. 200, and 60+ years Rs. 150. Read an age and print the ticket price.

SOLUTION

```
age = int(input("Enter age: "))

if age < 5:
    price = 0
elif age < 18:
    price = 100
elif age < 60:
    price = 200
else:
    price = 150

print("Ticket price: Rs.", price)
```

OUTPUT

```
Enter age: 65
Ticket price: Rs. 150
```

Concept: ordering elif conditions so each range is checked correctly.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 44. BMI Calculator

Read weight (kg) and height (m), compute BMI = weight / height², and print the category: Underweight (< 18.5), Normal (18.5–24.9), Overweight (25–29.9), or Obese (30+).

SOLUTION

```
weight = float(input("Enter weight in kg: "))
height = float(input("Enter height in metres: "))

bmi = weight / height ** 2

if bmi < 18.5:
    category = "Underweight"
elif bmi < 25:
    category = "Normal"
elif bmi < 30:
    category = "Overweight"
else:
    category = "Obese"

print(f"BMI = {bmi:.2f} ({category})")
```

OUTPUT

```
Enter weight in kg: 70
Enter height in metres: 1.75
BMI = 22.86 (Normal)
```

Concept: computing a value first, then classifying it with an elif ladder.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 45. Divisible by Both 5 and 11

Read a number and check whether it is divisible by both 5 and 11, by only one of them, or by neither.

SOLUTION

```
num = int(input("Enter a number: "))

by5 = num % 5 == 0      # True or False
by11 = num % 11 == 0

if by5 and by11:
    print(num, "is divisible by both 5 and 11")
elif by5 or by11:
    print(num, "is divisible by only one of 5 and 11")
else:
    print(num, "is divisible by neither 5 nor 11")
```

OUTPUT

```
Enter a number: 385
385 is divisible by both 5 and 11
```

Concept: storing conditions in boolean variables and combining them with and / or.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 46. Season from Month Number

Read a month number (1–12) and print the season: Winter (12, 1, 2), Spring (3–5), Summer (6–8), or Autumn (9–11). Handle invalid months.

SOLUTION

```
month = int(input("Enter month number (1-12): "))

if month in (12, 1, 2):
    print("Winter")
elif 3 ≤ month ≤ 5:      # chained comparison: month is
    print("Spring")      between 3 and 5
elif 6 ≤ month ≤ 8:
    print("Summer")
elif 9 ≤ month ≤ 11:
    print("Autumn")
else:
    print("Invalid month number")
```

OUTPUT

```
Enter month number (1-12): 7
Summer
```

Concept: chained comparisons ($a \leq x \leq b$) and membership with `in`.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 47. Triangle Type Checker

Read three side lengths. First check whether they can form a valid triangle (sum of any two sides must be greater than the third). If valid, print whether the triangle is Equilateral, Isosceles or Scalene.

SOLUTION

```
a, b, c = map(int, input("Enter three sides: ").split())

# triangle inequality: every pair of sides must outweigh
the third side
if a + b > c and b + c > a and a + c > b:
    if a == b == c:
        print("Equilateral triangle")
    elif a == b or b == c or a == c:
        print("Isosceles triangle")
    else:
        print("Scalene triangle")
else:
    print("Not a valid triangle")
```

OUTPUT

```
Enter three sides: 5 5 8
Isosceles triangle
```

Concept: nested if statements and validating input before classifying.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 48. Simple Calculator Using match-case

Read two numbers and an operator (+, -, *, /) and print the result using Python's match-case statement. Guard against division by zero.

SOLUTION

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
op = input("Enter operator (+ - * /): ")

match op:
    # match-case needs Python 3.10 or newer
    case "+":
        print("Result =", a + b)
    case "-":
        print("Result =", a - b)
    case "*":
        print("Result =", a * b)
    case "/":
        if b == 0:
            print("Error: cannot divide by zero")
        else:
            print("Result =", a / b)
    case _:
        # _ is the default, like else
        print("Unknown operator")
```

OUTPUT

```
Enter first number: 12
Enter second number: 4
Enter operator (+ - * /): /
Result = 3.0
```

Concept: structural pattern matching (match-case) with a wildcard default.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 49. Electricity Bill with Slabs

Compute an electricity bill: first 100 units at Rs. 1.50/unit, next 100 at Rs. 2.50/unit, next 100 at Rs. 4.00/unit, and anything above 300 units at Rs. 6.00/unit.

SOLUTION

```
units = int(input("Enter units consumed: "))

if units ≤ 100:
    bill = units * 1.5
elif units ≤ 200:
    bill = 100 * 1.5 + (units - 100) * 2.5
elif units ≤ 300:
    bill = 100 * 1.5 + 100 * 2.5 + (units - 200) * 4.0
else:
    bill = 100 * 1.5 + 100 * 2.5 + 100 * 4.0 + (units -
300) * 6.0

print(f"Bill for {units} units = Rs. {bill:.2f}")
```

OUTPUT

```
Enter units consumed: 250
Bill for 250 units = Rs. 600.00
```

Concept: slab-based (tiered) calculation where each slab is charged separately.

Section 3 · Conditionals (if / elif / else) (continued)

Problem 50. Income Tax Calculator with Slabs

Compute income tax: no tax up to Rs. 2,50,000; 5% on the next Rs. 2,50,000; 20% on the next Rs. 5,00,000; and 30% on income above Rs. 10,00,000. Print the total tax.

SOLUTION

```
income = float(input("Enter annual income: "))

tax = 0.0
if income > 1000000:
    tax += (income - 1000000) * 0.30    # 30% slab
    income = 1000000                  # remaining income
    falls in lower slabs
if income > 500000:
    tax += (income - 500000) * 0.20    # 20% slab
    income = 500000
if income > 250000:
    tax += (income - 250000) * 0.05    # 5% slab

print(f"Total tax = Rs. {tax:.2f}")
```

OUTPUT

```
Enter annual income: 1200000
Total tax = Rs. 172500.00
```

Concept: sequential if statements to apply progressive tax slabs from the top down.

SECTION 4

Loops (for / while)

(Problems 51–75)

Problem 51. Print Numbers 1 to 10

Write a program that prints the numbers from 1 to 10, each on its own line, using a for loop.

SOLUTION

```
for i in range(1, 11):    # range(1, 11) gives 1, 2, ...,
    print(i)              10
```

OUTPUT

```
1
2
3
4
5
6
7
8
9
10
```

Concept: basic for loop with range(start, stop).

Section 4 · Loops (for / while) (continued)

Problem 52. Multiplication Table

Read a number N from the user and print its multiplication table from 1 to 10.

SOLUTION

```
n = int(input("Enter a number: "))
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

OUTPUT

```
Enter a number: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Concept: for loop combined with f-string formatting.

Section 4 · Loops (for / while) (continued)

Problem 53. Sum of First N Natural Numbers

Read N from the user and compute $1 + 2 + 3 + \dots + N$ using a loop.

SOLUTION

```
n = int(input("Enter N: "))
total = 0
for i in range(1, n + 1):
    total += i          # same as total = total + i
print(f"Sum of first {n} natural numbers is {total}")
```

OUTPUT

```
Enter N: 10
Sum of first 10 natural numbers is 55
```

Concept: accumulator variable updated inside a loop.

Section 4 · Loops (for / while) (continued)

Problem 54. Reversed Counting

Print the numbers from 10 down to 1 on one line, separated by spaces, using a loop that counts backwards.

SOLUTION

```
for i in range(10, 0, -1):    # step of -1 counts down
    print(i, end=" ")
print()
```

OUTPUT

```
10 9 8 7 6 5 4 3 2 1
```

Concept: range with a negative step and print with end=" ".

Section 4 · Loops (for / while) (continued)

Problem 55. Factorial of a Number

Read a number N and compute $N!$ (N factorial), which is the product $1 \times 2 \times \dots \times N$.

SOLUTION

```
n = int(input("Enter a number: "))
factorial = 1
for i in range(1, n + 1):
    factorial *= i      # multiply the running product by i
print(f"{n}! = {factorial}")
```

OUTPUT

```
Enter a number: 5
5! = 120
```

Concept: running product accumulator inside a for loop.

Section 4 · Loops (for / while) (continued)

Problem 56. Table of Squares

Print a small table showing each number from 1 to 10 next to its square.

SOLUTION

```
print("Number  Square")
for i in range(1, 11):
    print(f"{i:<7} {i * i}")    # :<7 left-aligns i in 7
spaces
```

OUTPUT

Number	Square
1	1
2	4
3	9
4	16
5	25
6	36
7	49
8	64
9	81
10	100

Concept: for loop with aligned output using format specifiers.

Section 4 · Loops (for / while) (continued)

Problem 57. Sum of Even and Odd Separately

Read N and compute two totals for the numbers 1 to N: the sum of the even numbers and the sum of the odd numbers.

SOLUTION

```
n = int(input("Enter N: "))
even_sum = 0
odd_sum = 0
for i in range(1, n + 1):
    if i % 2 == 0:
        even_sum += i
    else:
        odd_sum += i
print(f"Even sum = {even_sum}")
print(f"Odd sum  = {odd_sum}")
```

OUTPUT

```
Enter N: 10
Even sum = 30
Odd sum  = 25
```

Concept: if/else branching inside a loop with two accumulators.

Section 4 · Loops (for / while) (continued)

Problem 58. Count Digits in a Number

Read a positive integer and count how many digits it has using a while loop (do not convert it to a string).

SOLUTION

```
n = int(input("Enter a number: "))
temp = n
count = 0
while temp > 0:
    temp //= 10      # drop the last digit
    count += 1
print(f"{n} has {count} digits")
```

OUTPUT

```
Enter a number: 45678
45678 has 5 digits
```

Concept: while loop with integer division to strip digits.

Section 4 · Loops (for / while) (continued)

Problem 59. Sum of Digits

Read a positive integer and compute the sum of its digits using a while loop.

SOLUTION

```
n = int(input("Enter a number: "))
temp = n
digit_sum = 0
while temp > 0:
    digit_sum += temp % 10    # % 10 gives the last digit
    temp //= 10
print(f"Sum of digits of {n} is {digit_sum}")
```

OUTPUT

```
Enter a number: 9875
Sum of digits of 9875 is 29
```

Concept: extracting digits with % 10 and // 10 in a while loop.

Section 4 · Loops (for / while) (continued)

Problem 60. Sum of Series $1 + 1/2 + \dots + 1/n$

Read n and compute the harmonic sum $1 + 1/2 + 1/3 + \dots + 1/n$, printed to 4 decimal places.

SOLUTION

```
n = int(input("Enter n: "))
total = 0.0
for i in range(1, n + 1):
    total += 1 / i
print(f"Sum of series = {total:.4f}")
```

OUTPUT

```
Enter n: 4
Sum of series = 2.0833
```

Concept: floating-point accumulation and .4f formatting.

Section 4 · Loops (for / while) (continued)

Problem 61. Reverse a Number

Read a positive integer and print it with its digits reversed, using arithmetic in a while loop.

SOLUTION

```
n = int(input("Enter a number: "))
temp = n
reversed_num = 0
while temp > 0:
    reversed_num = reversed_num * 10 + temp % 10
    temp //= 10
print(f"Reverse of {n} is {reversed_num}")
```

OUTPUT

```
Enter a number: 1234
Reverse of 1234 is 4321
```

Concept: building a number digit by digit with multiply-by-10.

Section 4 · Loops (for / while) (continued)

Problem 62. Palindrome Number

Read a positive integer and check whether it is a palindrome, i.e. it reads the same forwards and backwards.

SOLUTION

```
n = int(input("Enter a number: "))
temp = n
reversed_num = 0
while temp > 0:
    reversed_num = reversed_num * 10 + temp % 10
    temp //= 10
if reversed_num == n:
    print(f"{n} is a palindrome")
else:
    print(f"{n} is not a palindrome")
```

OUTPUT

```
Enter a number: 12321
12321 is a palindrome
```

Concept: number reversal plus comparison with the original.

Section 4 · Loops (for / while) (continued)

Problem 63. Fibonacci Series

Read N and print the first N terms of the Fibonacci series, where each term is the sum of the previous two (starting 0, 1).

SOLUTION

```
n = int(input("How many terms? "))
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b    # shift the pair forward by one term
print()
```

OUTPUT

```
How many terms? 8
0 1 1 2 3 5 8 13
```

Concept: tuple swapping to carry two previous values through a loop.

Section 4 · Loops (for / while) (continued)

Problem 64. Prime Number Check

Read a number and check whether it is prime, i.e. greater than 1 and divisible only by 1 and itself.

SOLUTION

```
n = int(input("Enter a number: "))
is_prime = n > 1
for i in range(2, int(n ** 0.5) + 1):    # check up to the
    square root
    if n % i == 0:
        is_prime = False
        break                          # one divisor is
enough to decide
if is_prime:
    print(f"{n} is prime")
else:
    print(f"{n} is not prime")
```

OUTPUT

```
Enter a number: 29
29 is prime
```

Concept: trial division up to the square root with an early break.

Section 4 · Loops (for / while) (continued)

Problem 65. Primes in a Range

Read two numbers, low and high, and print every prime number between them (inclusive).

SOLUTION

```
low = int(input("Enter low: "))
high = int(input("Enter high: "))
for n in range(max(low, 2), high + 1):
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            break          # n is not prime, stop
    else:                  # runs only if the inner loop
        print(n, end=" ")  # never broke
print()
```

OUTPUT

```
Enter low: 10
Enter high: 30
11 13 17 19 23 29
```

Concept: nested loops and Python's for-else clause.

Section 4 · Loops (for / while) (continued)

Problem 66. Armstrong Number

Read a 3-digit number and check whether it is an Armstrong number, i.e. equal to the sum of the cubes of its digits (e.g. $153 = 1^3 + 5^3 + 3^3$).

SOLUTION

```
n = int(input("Enter a 3-digit number: "))
temp = n
total = 0
while temp > 0:
    digit = temp % 10
    total += digit ** 3
    temp //= 10
if total == n:
    print(f"{n} is an Armstrong number")
else:
    print(f"{n} is not an Armstrong number")
```

OUTPUT

```
Enter a 3-digit number: 153
153 is an Armstrong number
```

Concept: digit extraction in a while loop with a power-sum check.

Section 4 · Loops (for / while) (continued)

Problem 67. Perfect Number

Read a number and check whether it is perfect, i.e. equal to the sum of its proper divisors (e.g. $28 = 1 + 2 + 4 + 7 + 14$).

SOLUTION

```
n = int(input("Enter a number: "))
divisor_sum = 0
for i in range(1, n):          # proper divisors exclude n
    if n % i == 0:
        divisor_sum += i
if divisor_sum == n:
    print(f"{n} is a perfect number")
else:
    print(f"{n} is not a perfect number")
```

OUTPUT

```
Enter a number: 28
28 is a perfect number
```

Concept: finding divisors with the modulo operator in a loop.

Section 4 · Loops (for / while) (continued)

Problem 68. Strong Number

Read a number and check whether it is a strong number, i.e. equal to the sum of the factorials of its digits (e.g. $145 = 1! + 4! + 5!$).

SOLUTION

```
n = int(input("Enter a number: "))
temp = n
total = 0
while temp > 0:
    digit = temp % 10
    fact = 1
    for i in range(1, digit + 1):    # factorial of this
digit
        fact *= i
    total += fact
    temp //= 10
if total == n:
    print(f"{n} is a strong number")
else:
    print(f"{n} is not a strong number")
```

OUTPUT

```
Enter a number: 145
145 is a strong number
```

Concept: a for loop (factorial) nested inside a while loop (digits).

Section 4 · Loops (for / while) (continued)

Problem 69. GCD with a While Loop

Read two numbers and compute their greatest common divisor using the Euclidean algorithm with a while loop.

SOLUTION

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
x, y = a, b
while y != 0:
    x, y = y, x % y    # replace with (smaller, remainder)
    each round
print(f"GCD of {a} and {b} is {x}")
```

OUTPUT

```
Enter first number: 48
Enter second number: 36
GCD of 48 and 36 is 12
```

Concept: Euclidean algorithm driven by a while loop condition.

Section 4 · Loops (for / while) (continued)

Problem 70. Decimal to Binary

Read a decimal number and convert it to binary using repeated division by 2 (do not use `bin()`).

SOLUTION

```
n = int(input("Enter a decimal number: "))
temp = n
binary = ""
while temp > 0:
    binary = str(temp % 2) + binary    # remainders build
    up right to left
    temp //= 2
print(f"Binary of {n} is {binary}")
```

OUTPUT

```
Enter a decimal number: 13
Binary of 13 is 1101
```

Concept: repeated division by 2, collecting remainders in reverse.

Section 4 · Loops (for / while) (continued)

Problem 71. Binary to Decimal

Read a binary number as a string and convert it to decimal by processing one bit at a time (do not use `int(s, 2)`).

SOLUTION

```
bits = input("Enter a binary number: ")
decimal = 0
for bit in bits:
    decimal = decimal * 2 + int(bit)    # shift left, then
    add the new bit
print(f"Decimal value: {decimal}")
```

OUTPUT

```
Enter a binary number: 1101
Decimal value: 13
```

Concept: positional number systems via multiply-and-add in a loop.

Section 4 · Loops (for / while) (continued)

Problem 72. Digital Root

Read a number and repeatedly replace it with the sum of its digits until a single digit remains, printing each step.

SOLUTION

```
n = int(input("Enter a number: "))
while n ≥ 10:                # keep going until one digit
    is left
    s = 0
    temp = n
    while temp > 0:
        s += temp % 10
        temp //= 10
    n = s
    print("Step:", n)
print(f"Digital root: {n}")
```

OUTPUT

```
Enter a number: 9875
Step: 29
Step: 11
Step: 2
Digital root: 2
```

Concept: nested while loops repeating a digit-sum until a condition holds.

Section 4 · Loops (for / while) (continued)

Problem 73. Collatz Sequence

Read a number and print its Collatz sequence: halve it if even, otherwise compute $3n + 1$, repeating until it reaches 1. Also print how many steps it took.

SOLUTION

```
n = int(input("Enter a number: "))
steps = 0
while n != 1:
    print(n, end=" ")
    if n % 2 == 0:
        n //= 2
    else:
        n = 3 * n + 1
    steps += 1
print(1)
print(f"Steps taken: {steps}")
```

OUTPUT

```
Enter a number: 6
6 3 10 5 16 8 4 2 1
Steps taken: 8
```

Concept: while loop whose exit depends on a computed value, with a step counter.

Section 4 · Loops (for / while) (continued)

Problem 74. Number Guessing Loop

A secret number is stored in the program. Keep asking the user for guesses in a while loop, saying "Too low!" or "Too high!" after each wrong guess, and report the number of tries when they get it right.

SOLUTION

```
secret = 7
attempts = 0
guess = -1                                # start with a value that
cannot match
while guess != secret:
    guess = int(input("Guess the number (1-10): "))
    attempts += 1
    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
print(f"Correct! You took {attempts} tries.")
```

OUTPUT

```
Guess the number (1-10): 3
Too low!
Guess the number (1-10): 9
Too high!
Guess the number (1-10): 7
Correct! You took 3 tries.
```

Concept: sentinel-controlled while loop reading input until a condition is met.

Section 4 · Loops (for / while) (continued)

Problem 75. Nested Loop Coordinate Pairs

Use two nested loops to print all coordinate pairs (row, col) of a 3×3 grid, one row of pairs per line.

SOLUTION

```
size = 3
for row in range(size):
    for col in range(size):          # inner loop runs fully
        # for each row
        print(f"({row},{col})", end=" ")
    print()                         # newline after each
    row
```

OUTPUT

```
(0,0) (0,1) (0,2)
(1,0) (1,1) (1,2)
(2,0) (2,1) (2,2)
```

Concept: nested for loops generating every combination of two ranges.

SECTION 5

Star & Number Patterns

(Problems 76–90)

Problem 76. Right Triangle of Stars

Print a right-angled triangle of stars with 5 rows, where row *i* contains *i* stars.

SOLUTION

```
rows = 5
for i in range(1, rows + 1):
    print("*" * i)      # string repetition builds each row
```

OUTPUT

```
*
**
***
****
*****
```

Concept: loop variable controlling string repetition.

Section 5 · Star & Number Patterns (continued)

Problem 77. Inverted Right Triangle

Print an inverted right-angled triangle of stars with 5 rows, starting with 5 stars and shrinking by one star per row.

SOLUTION

```
rows = 5
for i in range(rows, 0, -1):    # count down from 5 to 1
    print("*" * i)
```

OUTPUT

```
*****
****
***
**
*
```

Concept: descending range driving string repetition.

Section 5 · Star & Number Patterns (continued)

Problem 78. Right-Aligned Triangle

Print a triangle of 5 rows where the stars are pushed to the right side using leading spaces.

SOLUTION

```
rows = 5
for i in range(1, rows + 1):
    print(" " * (rows - i) + "*" * i)    # spaces first,
    then stars
```

OUTPUT

```
    *
   **
  ***
 ****
*****
```

Concept: combining leading spaces and stars for alignment.

Section 5 · Star & Number Patterns (continued)

Problem 79. Number Triangle

Print a 5-row triangle where row *i* contains the numbers 1 to *i* joined together (1, 12, 123, ...).

SOLUTION

```
rows = 5
for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(j, end="")
    print()
```

OUTPUT

```
1
12
123
1234
12345
```

Concept: nested loops where the inner limit depends on the outer variable.

Section 5 · Star & Number Patterns (continued)

Problem 80. Alphabet Pattern

Print a 5-row pattern where row *i* contains the first *i* capital letters (A, AB, ABC, ...). Use `chr()` and `ord()` to generate letters.

SOLUTION

```
rows = 5
for i in range(rows):
    for j in range(i + 1):
        print(chr(ord("A") + j), end="")    # ord("A") is
65
    print()
```

OUTPUT

```
A
AB
ABC
ABCD
ABCDE
```

Concept: converting between characters and codes with `chr()` and `ord()`.

Section 5 · Star & Number Patterns (continued)

Problem 81. Floyd's Triangle

Print Floyd's triangle with 5 rows: consecutive natural numbers filled row by row, with row *i* holding *i* numbers.

SOLUTION

```
rows = 5
num = 1
for i in range(1, rows + 1):
    for j in range(i):
        print(num, end=" ")
        num += 1                # keep counting across rows
    print()
```

OUTPUT

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Concept: a counter shared across nested loops.

Section 5 · Star & Number Patterns (continued)

Problem 82. Pyramid

Print a centered pyramid of stars with 5 rows, where row i has $2i - 1$ stars.

SOLUTION

```
rows = 5
for i in range(1, rows + 1):
    spaces = " " * (rows - i)
    stars = "*" * (2 * i - 1)    # odd number of stars
    keeps it centered
    print(spaces + stars)
```

OUTPUT

```
  *
 ***
*****
*****
*****
```

Concept: the $2i - 1$ odd-count formula for centered patterns.

Section 5 • Star & Number Patterns (continued)

Problem 83. Inverted Pyramid

Print an upside-down centered pyramid of stars with 5 rows, starting from the widest row.

SOLUTION

```
rows = 5
for i in range(rows, 0, -1):
    print(" " * (rows - i) + "*" * (2 * i - 1))
```

OUTPUT

* * * * *

* * * * *

* * * * *

* * *

*

Concept: reversing the pyramid logic with a descending range.

Section 5 · Star & Number Patterns (continued)

Problem 84. Hollow Square

Print a 5×5 hollow square: stars on the border, spaces inside.

SOLUTION

```
size = 5
for i in range(size):
    for j in range(size):
        # border rows/columns get a star, everything else
        # a space
        if i == 0 or i == size - 1 or j == 0 or j == size - 1:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

OUTPUT

```
*****
*      *
*      *
*      *
*      *
*****
```

Concept: boundary conditions inside nested loops.

Section 5 · Star & Number Patterns (continued)

Problem 85. Hollow Right Triangle

Print a hollow right-angled triangle with 5 rows: stars only on the two sides and the base.

SOLUTION

```
rows = 5
for i in range(1, rows + 1):
    for j in range(1, i + 1):
        # first column, diagonal, or last row gets a star
        if j == 1 or j == i or i == rows:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

OUTPUT

```
*
**
* *
* *
*****
```

Concept: printing only edge positions of a triangle.

Section 5 · Star & Number Patterns (continued)

Problem 86. Diamond

Print a diamond of stars with a widest row of 9 stars: a 5-row pyramid on top followed by a 4-row inverted pyramid.

SOLUTION

```
n = 5
for i in range(1, n + 1):          # top half (including
    print(" " * (n - i) + "*" * (2 * i - 1))    middle row)
for i in range(n - 1, 0, -1):      # bottom half
    print(" " * (n - i) + "*" * (2 * i - 1))
```

OUTPUT

```
  *
 ***
*****
*****
*****
*****
 *****
  *****
   *****
    ***
     *
```

Concept: combining an ascending and a descending loop to mirror a shape.

Section 5 · Star & Number Patterns (continued)

Problem 87. Number Pyramid (Palindromic Rows)

Print a 5-row pyramid where row *i* shows the numbers 1 up to *i* and back down to 1 (e.g. row 3 is 12321), centered with spaces.

SOLUTION

```
rows = 5
for i in range(1, rows + 1):
    print(" " * (rows - i), end="")
    for j in range(1, i + 1):        # counting up: 1 .. i
        print(j, end="")
    for j in range(i - 1, 0, -1):    # counting down: i-1
        .. 1
        print(j, end="")
    print()
```

OUTPUT

```
  1
 121
12321
1234321
123454321
```

Concept: two inner loops (ascending and descending) building a palindromic row.

Section 5 · Star & Number Patterns (continued)

Problem 88. Pascal's Triangle

Print the first 5 rows of Pascal's triangle, where each entry is computed from the previous entry in the same row, and rows are centered.

SOLUTION

```
n = 5
for i in range(n):
    val = 1
    row = []
    for j in range(i + 1):
        row.append(str(val))
        val = val * (i - j) // (j + 1)    # next binomial
    coefficient
    print(" ".join(row).center(2 * n - 1))
```

OUTPUT

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

Concept: computing binomial coefficients iteratively and centering strings.

Section 5 · Star & Number Patterns (continued)

Problem 89. Butterfly Pattern

Print a butterfly pattern of stars with wing height 4: in the top half, row i has i stars, a gap, then i stars; the bottom half mirrors the top.

SOLUTION

```
n = 4
for i in range(1, n + 1):          # top half
    print("*" * i + " " * (2 * (n - i)) + "*" * i)
for i in range(n, 0, -1):         # bottom half
    (mirror)
    print("*" * i + " " * (2 * (n - i)) + "*" * i)
```

OUTPUT

```
*      *
**     **
***    ***
*****
*****
***    ***
**     **
*      *
```

Concept: symmetric pattern built from stars, a computed gap, and mirroring.

Section 5 · Star & Number Patterns (continued)

Problem 90. Zig-Zag Pattern

Print a 3-row zig-zag (wave) pattern of stars that is 9 columns wide, placing a star wherever the row and column positions satisfy the wave conditions.

SOLUTION

```
rows, cols = 3, 9
for i in range(1, rows + 1):
    for j in range(1, cols + 1):
        # stars sit on the diagonals of the wave or the
        # middle row anchors
        if (i + j) % 4 == 0 or (i == 2 and j % 4 == 0):
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

OUTPUT

```
  *   *
 * * * *
*     *   *
```

Concept: modulo-based position conditions inside nested loops.

SECTION 6

Strings

(Problems 91–115)

Problem 91. Reverse a String

Read a string from the user and print it in reverse order.

SOLUTION

```
text = input("Enter a string: ")
reversed_text = text[::-1] # slice with step -1 walks the
string backwards
print("Reversed:", reversed_text)
```

OUTPUT

```
Enter a string: hello
Reversed: olleh
```

Concept: string slicing with a negative step.

Section 6 · Strings (continued)

Problem 92. Count Vowels

Count how many vowels (a, e, i, o, u) appear in a string, ignoring case.

SOLUTION

```
text = input("Enter a string: ")
count = 0
for ch in text.lower():
    if ch in "aeiou":
        count += 1
print("Vowels:", count)
```

OUTPUT

```
Enter a string: programming
Vowels: 3
```

Concept: looping over characters and membership testing with `in`.

Section 6 · Strings (continued)

Problem 93. Count Consonants

Count the consonants in a string. Only letters count, so digits, spaces and symbols are skipped.

SOLUTION

```
text = input("Enter a string: ")
count = 0
for ch in text.lower():
    if ch.isalpha() and ch not in "aeiou":
        count += 1
print("Consonants:", count)
```

OUTPUT

```
Enter a string: hello world
Consonants: 7
```

Concept: combining `isalpha()` with membership tests.

Section 6 · Strings (continued)

Problem 94. Palindrome Check

Check whether a string reads the same forwards and backwards (a palindrome).

SOLUTION

```
text = input("Enter a string: ").lower()
if text == text[::-1]:
    print("It is a palindrome")
else:
    print("Not a palindrome")
```

OUTPUT

```
Enter a string: madam
It is a palindrome
```

Concept: comparing a string with its reversed slice.

Section 6 · Strings (continued)

Problem 95. Count Words in a Sentence

Read a sentence and count how many words it contains.

SOLUTION

```
sentence = input("Enter a sentence: ")
words = sentence.split() # split() with no argument
                        # handles extra spaces too
print("Word count:", len(words))
```

OUTPUT

```
Enter a sentence: Python is fun to learn
Word count: 5
```

Concept: `split()` to break a sentence into a list of words.

Problem 96. Swap Case Without `swapcase()`

Convert every uppercase letter to lowercase and every lowercase letter to uppercase, without using `swapcase()`.

SOLUTION

```
text = input("Enter a string: ")
result = ""
for ch in text:
    if ch.isupper():
        result += ch.lower()
    elif ch.islower():
        result += ch.upper()
    else:
        result += ch # keep spaces, digits, symbols unchanged
print("Swapped:", result)
```

OUTPUT

```
Enter a string: Hello World
Swapped: hELLO wORLD
```

Concept: character classification with `isupper()` / `islower()` and string building.

Section 6 · Strings (continued)

Problem 97. Count Uppercase and Lowercase Letters

Count how many uppercase letters and how many lowercase letters a string contains.

SOLUTION

```
text = input("Enter a string: ")
upper = 0
lower = 0
for ch in text:
    if ch.isupper():
        upper += 1
    elif ch.islower():
        lower += 1
print("Uppercase:", upper)
print("Lowercase:", lower)
```

OUTPUT

```
Enter a string: PyTh0n Rocks
Uppercase: 4
Lowercase: 7
```

Concept: counters updated inside a character loop.

Section 6 · Strings (continued)

Problem 98. Remove All Spaces

Remove every space character from a string without using `replace()`.

SOLUTION

```
text = input("Enter a string: ")
result = ""
for ch in text:
    if ch != " ":
        result += ch
print("Without spaces:", result)
```

OUTPUT

```
Enter a string: py thon is fun
Without spaces: pythonisfun
```

Concept: filtering characters while rebuilding a string.

Problem 99. Check Substring Without Using in

Check whether one string appears inside another, without using the `in` operator or `find()`. Print the index where it starts, or a not-found message.

SOLUTION

```
text = "programming"
sub = "gram"
found = -1
# try every possible starting position
for i in range(len(text) - len(sub) + 1):
    if text[i:i + len(sub)] == sub:
        found = i
        break
if found == -1:
    print("Substring not found")
else:
    print("Found at index", found)
```

OUTPUT

```
Found at index 3
```

Concept: sliding-window comparison over every starting index.

Problem 100. Replace a Substring Without replace()

Replace every occurrence of one word with another in a string, without using `replace()`.

SOLUTION

```
text = "the cat sat on the cat mat"
old = "cat"
new = "dog"
result = ""
i = 0
while i < len(text):
    if text[i:i + len(old)] == old:
        result += new
        i += len(old) # jump past the matched word
    else:
        result += text[i]
        i += 1
print(result)
```

OUTPUT

```
the dog sat on the dog mat
```

Concept: manual scanning with a while loop and slice comparison.

Section 6 · Strings (continued)

Problem 101. Capitalize Each Word Without .title()

Capitalize the first letter of every word in a sentence, without using `title()` or `capitalize()`.

SOLUTION

```
sentence = input("Enter a sentence: ")
words = sentence.split()
capitalized = []
for word in words:
    capitalized.append(word[0].upper() + word[1:])
print(" ".join(capitalized))
```

OUTPUT

```
Enter a sentence: hello world from python
Hello World From Python
```

Concept: split, transform each word with slicing, then join.

Section 6 · Strings (continued)

Problem 102. Character Frequency

Count how many times each character appears in a string and print the counts.

SOLUTION

```
text = input("Enter a string: ")
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1 # get() returns 0 if
    the key is new
for ch, count in freq.items():
    print(ch, "→", count)
```

OUTPUT

```
Enter a string: banana
b → 1
a → 3
n → 2
```

Concept: dictionary counting with `dict.get()`.

Problem 103. Remove Duplicate Characters

Remove duplicate characters from a string, keeping only the first occurrence of each character.

SOLUTION

```
text = input("Enter a string: ")
result = ""
for ch in text:
    if ch not in result: # only add characters we have
        not seen yet
        result += ch
print("Without duplicates:", result)
```

OUTPUT

```
Enter a string: programming
Without duplicates: progamin
```

Concept: building a result string while checking membership to skip repeats.

Problem 104. Anagram Check

Check whether two strings are anagrams, meaning they contain exactly the same letters in a different order.

SOLUTION

```
s1 = input("Enter first word: ").lower()
s2 = input("Enter second word: ").lower()
if sorted(s1) == sorted(s2): # same letters sort into the
    same sequence
    print("They are anagrams")
else:
    print("Not anagrams")
```

OUTPUT

```
Enter first word: listen
Enter second word: silent
They are anagrams
```

Concept: `sorted()` as a canonical form for comparing letter contents.

Section 6 · Strings (continued)

Problem 105. Longest Word in a Sentence

Find and print the longest word in a sentence.

SOLUTION

```
sentence = input("Enter a sentence: ")
longest = ""
for word in sentence.split():
    if len(word) > len(longest):
        longest = word
print("Longest word:", longest)
```

OUTPUT

```
Enter a sentence: I love programming in Python
Longest word: programming
```

Concept: tracking a running maximum while looping over words.

Section 6 · Strings (continued)

Problem 106. Reverse the Word Order

Reverse the order of words in a sentence. The letters inside each word stay the same.

SOLUTION

```
sentence = input("Enter a sentence: ")
words = sentence.split()
reversed_words = words[::-1]
print(" ".join(reversed_words))
```

OUTPUT

```
Enter a sentence: Python is easy
easy is Python
```

Concept: split, reverse the list with slicing, then join.

Problem 107. Acronym Generator

Build an acronym from a phrase by taking the first letter of each word in uppercase.

SOLUTION

```
phrase = input("Enter a phrase: ")
acronym = ""
for word in phrase.split():
    acronym += word[0].upper()
print("Acronym:", acronym)
```

OUTPUT

```
Enter a phrase: random access memory
Acronym: RAM
```

Concept: indexing the first character of each word.

Problem 108. First Non-Repeating Character

Find the first character in a string that appears exactly once.

SOLUTION

```
text = input("Enter a string: ")
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
answer = None
for ch in text: # second pass keeps the original order
    if freq[ch] == 1:
        answer = ch
        break
if answer is None:
    print("No non-repeating character")
else:
    print("First non-repeating character:", answer)
```

OUTPUT

```
Enter a string: swiss
First non-repeating character: w
```

Concept: two-pass technique — count first, then scan in order.

Section 6 · Strings (continued)

Problem 109. Pangram Check

Check whether a sentence is a pangram, meaning it uses every letter of the English alphabet at least once.

SOLUTION

```
sentence = input("Enter a sentence: ").lower()
letters = set()
for ch in sentence:
    if ch.isalpha():
        letters.add(ch)
if len(letters) == 26:
    print("It is a pangram")
else:
    print("Not a pangram")
```

OUTPUT

```
Enter a sentence: the quick brown fox jumps over the lazy
dog
It is a pangram
```

Concept: using a set to collect unique letters.

Section 6 · Strings (continued)

Problem 110. Remove Punctuation

Remove all punctuation from a string, keeping only letters, digits and spaces.

SOLUTION

```
text = input("Enter a string: ")
result = ""
for ch in text:
    if ch.isalnum() or ch == " ":
        result += ch
print("Cleaned:", result)
```

OUTPUT

```
Enter a string: hello, world! how's it going?
Cleaned: hello world hows it going
```

Concept: `isalnum()` to keep only letters and digits.

Problem 111. Count Substring Occurrences (Overlapping)

Count how many times a substring occurs in a string, including overlapping matches. For example, "aa" occurs 3 times in "aaaa".

SOLUTION

```
text = "aaaa"
sub = "aa"
count = 0
for i in range(len(text) - len(sub) + 1):
    if text[i:i + len(sub)] == sub:
        count += 1 # note: str.count() would miss overlaps
print("Occurrences:", count)
```

OUTPUT

```
Occurrences: 3
```

Concept: checking every start index so overlapping matches are counted.

Problem 112. String Compression

Compress a string by replacing each run of repeated characters with the character followed by its count. For example, "aaabb" becomes "a3b2".

SOLUTION

```
text = input("Enter a string: ")
result = ""
i = 0
while i < len(text):
    count = 1
    while i + count < len(text) and text[i + count] == text[i]:
        count += 1 # extend the run of equal characters
    result += text[i] + str(count)
    i += count # jump past the whole run
print("Compressed:", result)
```

OUTPUT

```
Enter a string: aaabbbbcc
Compressed: a3b4c2
```

Concept: run-length encoding with a nested while loop.

Section 6 · Strings (continued)

Problem 113. Caesar Cipher

Encrypt a message by shifting every letter forward by a given number of places in the alphabet, wrapping around from z to a. Non-letters stay unchanged.

SOLUTION

```
text = input("Enter a message: ")
shift = int(input("Enter shift: "))
result = ""
for ch in text:
    if ch.isupper():
        # ord() gives the character code; % 26 wraps z
        back to a
        result += chr((ord(ch) - ord("A") + shift) % 26 +
ord("A"))
    elif ch.islower():
        result += chr((ord(ch) - ord("a") + shift) % 26 +
ord("a"))
    else:
        result += ch
print("Encrypted:", result)
```

OUTPUT

```
Enter a message: Hello, World!
Enter shift: 3
Encrypted: Khoor, Zruog!
```

Concept: `ord()` / `chr()` arithmetic with modulo wrap-around.

Problem 114. Check String Rotation

Check whether one string is a rotation of another. For example, "cdeab" is a rotation of "abcde".

SOLUTION

```
s1 = "abcde"
s2 = "cdeab"
# every rotation of s1 appears inside s1 repeated twice
if len(s1) == len(s2) and s2 in s1 + s1:
    print(s2, "is a rotation of", s1)
else:
    print(s2, "is not a rotation of", s1)
```

OUTPUT

```
cdeab is a rotation of abcde
```

Concept: the doubled-string trick for rotation checks.

Problem 115. Longest Common Prefix

Given a list of strings, find the longest prefix shared by all of them. Print an empty-prefix message if there is none.

SOLUTION

```
words = ["flower", "flow", "flight"]
prefix = words[0]
for word in words[1:]:
    # shrink the prefix until the current word starts with
    it
    while not word.startswith(prefix):
        prefix = prefix[:-1]
        if prefix == "":
            break
if prefix:
    print("Longest common prefix:", prefix)
else:
    print("No common prefix")
```

OUTPUT

```
Longest common prefix: fl
```

Concept: iteratively shrinking a candidate prefix with `startswith()`.

Lists

(Problems 116–140)

Problem 116. Maximum Without max()

Find the largest number in a list without using the built-in `max()` function.

SOLUTION

```
numbers = [12, 45, 2, 67, 33]
largest = numbers[0]
for n in numbers:
    if n > largest:
        largest = n
print("Largest:", largest)
```

OUTPUT

```
Largest: 67
```

Concept: tracking a running maximum in a loop.

Section 7 · Lists (continued)

Problem 117. Minimum Without min()

Find the smallest number in a list without using the built-in `min()` function.

SOLUTION

```
numbers = [12, 45, 2, 67, 33]
smallest = numbers[0]
for n in numbers:
    if n < smallest:
        smallest = n
print("Smallest:", smallest)
```

OUTPUT

```
Smallest: 2
```

Concept: tracking a running minimum in a loop.

Problem 118. Sum and Average

Compute the sum and the average of a list of numbers without using `sum()`.

SOLUTION

```
numbers = [10, 20, 30, 40]
total = 0
for n in numbers:
    total += n
average = total / len(numbers)
print("Sum:", total)
print("Average:", average)
```

OUTPUT

```
Sum: 100
Average: 25.0
```

Concept: accumulator pattern for totals.

Section 7 · Lists (continued)

Problem 119. Linear Search

Search for a target value in a list and print the index of its first occurrence, or -1 if it is not present.

SOLUTION

```
numbers = [4, 7, 1, 9, 3]
target = 9
index = -1
for i in range(len(numbers)):
    if numbers[i] == target:
        index = i
        break # stop at the first match
print("Index of", target, ":", index)
```

OUTPUT

```
Index of 9 : 3
```

Concept: linear search with early exit using `break`.

Problem 120. Count Occurrences of a Value

Count how many times a given value appears in a list, without using `list.count()`.

SOLUTION

```
numbers = [1, 2, 3, 2, 4, 2]
target = 2
count = 0
for n in numbers:
    if n == target:
        count += 1
print(target, "appears", count, "times")
```

OUTPUT

```
2 appears 3 times
```

Concept: conditional counting inside a loop.

Problem 121. List of Squares

Given a list of numbers, build a new list containing the square of each number.

SOLUTION

```
numbers = [1, 2, 3, 4, 5]
squares = [n ** 2 for n in numbers] # list comprehension
print("Squares:", squares)
```

OUTPUT

```
Squares: [1, 4, 9, 16, 25]
```

Concept: list comprehension for transforming a list.

Problem 122. Reverse a List In Place

Reverse a list in place by swapping elements, without using `reverse()` or slicing.

SOLUTION

```
numbers = [1, 2, 3, 4, 5]
left = 0
right = len(numbers) - 1
while left < right:
    numbers[left], numbers[right] = numbers[right],
    numbers[left]
    left += 1
    right -= 1
print("Reversed:", numbers)
```

OUTPUT

```
Reversed: [5, 4, 3, 2, 1]
```

Concept: two-pointer swapping from both ends.

Problem 123. Check If a List Is Sorted

Check whether a list is sorted in non-decreasing order.

SOLUTION

```
def is_sorted(nums):
    for i in range(len(nums) - 1):
        if nums[i] > nums[i + 1]: # one bad pair is
            enough to fail
            return False
    return True

print(is_sorted([1, 3, 5, 7]))
print(is_sorted([4, 2, 9]))
```

OUTPUT

```
True
False
```

Concept: comparing adjacent pairs.

Problem 124. Second Largest Element

Find the second largest distinct value in a list without sorting it.

SOLUTION

```
numbers = [10, 40, 30, 40, 20]
largest = None
second = None
for n in numbers:
    if largest is None or n > largest:
        second = largest # old largest becomes second
        largest = n
    elif n != largest and (second is None or n > second):
        second = n
print("Second largest:", second)
```

OUTPUT

```
Second largest: 30
```

Concept: tracking two running maximums in one pass.

Problem 125. Remove Duplicates (Keep Order)

Remove duplicate values from a list while keeping the original order of first appearances.

SOLUTION

```
numbers = [1, 2, 2, 3, 1, 4]
unique = []
for n in numbers:
    if n not in unique:
        unique.append(n)
print("Unique:", unique)
```

OUTPUT

```
Unique: [1, 2, 3, 4]
```

Concept: membership check before appending (set() alone would lose order).

Section 7 · Lists (continued)

Problem 126. Separate Even and Odd Numbers

Split a list of numbers into two lists: one with the even numbers and one with the odd numbers.

SOLUTION

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = []
odds = []
for n in numbers:
    if n % 2 == 0:
        evens.append(n)
    else:
        odds.append(n)
print("Evens:", evens)
print("Odds:", odds)
```

OUTPUT

```
Evens: [2, 4, 6, 8, 10]
Odds: [1, 3, 5, 7, 9]
```

Concept: partitioning a list with the modulo operator.

Problem 127. Cumulative Sum

Build a new list where each element is the sum of all elements up to that position. For [1, 2, 3, 4] the answer is [1, 3, 6, 10].

SOLUTION

```
numbers = [1, 2, 3, 4]
running = 0
cumulative = []
for n in numbers:
    running += n
    cumulative.append(running)
print("Cumulative sum:", cumulative)
```

OUTPUT

```
Cumulative sum: [1, 3, 6, 10]
```

Concept: prefix sums with a running total.

Problem 128. Remove All Occurrences of a Value

Remove every occurrence of a given value from a list.

SOLUTION

```
numbers = [1, 2, 3, 2, 4, 2]
value = 2
result = [n for n in numbers if n != value] # keep
everything except the value
print("After removal:", result)
```

OUTPUT

```
After removal: [1, 3, 4]
```

Concept: filtering with a list comprehension (safer than removing while looping).

Problem 129. Common Elements (Intersection)

Find the elements that appear in both of two lists, keeping the order of the first list.

SOLUTION

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]
common = []
for x in a:
    if x in b and x not in common:
        common.append(x)
print("Common elements:", common)
```

OUTPUT

```
Common elements: [3, 4]
```

Concept: intersection via membership testing.

Problem 130. Union of Two Lists

Combine two lists into one list of all distinct elements, keeping order of first appearance.

SOLUTION

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]
union = []
for x in a + b: # walk through both lists in order
    if x not in union:
        union.append(x)
print("Union:", union)
```

OUTPUT

```
Union: [1, 2, 3, 4, 5, 6]
```

Concept: concatenation plus duplicate filtering.

Problem 131. Find Duplicate Elements

Find every value that appears more than once in a list. Report each duplicate only once.

SOLUTION

```
numbers = [1, 2, 3, 2, 4, 1, 5]
seen = set()
duplicates = []
for n in numbers:
    if n in seen and n not in duplicates:
        duplicates.append(n) # second time we meet n
    seen.add(n)
print("Duplicates:", duplicates)
```

OUTPUT

```
Duplicates: [2, 1]
```

Concept: a `set` of seen values gives fast duplicate detection.

Problem 132. Find the Missing Number

A list contains the numbers 1 to n with exactly one number missing. Find the missing number.

SOLUTION

```
numbers = [1, 2, 4, 5, 6]
n = 6
expected = n * (n + 1) // 2 # sum of 1..n formula
actual = sum(numbers)
print("Missing number:", expected - actual)
```

OUTPUT

```
Missing number: 3
```

Concept: arithmetic-series formula compared with the actual sum.

Problem 133. Move Zeros to the End

Move all zeros in a list to the end while keeping the order of the non-zero elements.

SOLUTION

```
numbers = [0, 1, 0, 3, 12]
non_zero = [n for n in numbers if n != 0]
zero_count = len(numbers) - len(non_zero)
result = non_zero + [0] * zero_count # pad zeros at the
end
print("Result:", result)
```

OUTPUT

```
Result: [1, 3, 12, 0, 0]
```

Concept: stable partition using filtering and list repetition.

Problem 134. Rotate a List by k

Rotate a list to the right by k positions. For [1, 2, 3, 4, 5] and k = 2 the answer is [4, 5, 1, 2, 3].

SOLUTION

```
numbers = [1, 2, 3, 4, 5]
k = 2
k = k % len(numbers) # handles k larger than the list
length
rotated = numbers[-k:] + numbers[:-k]
print("Rotated:", rotated)
```

OUTPUT

```
Rotated: [4, 5, 1, 2, 3]
```

Concept: rotation by slicing and concatenation, with modulo normalization.

Section 7 · Lists (continued)

Problem 135. Chunk a List into Size-n Pieces

Split a list into consecutive chunks of size n. The final chunk may be smaller.

SOLUTION

```
numbers = [1, 2, 3, 4, 5, 6, 7]
size = 3
chunks = []
for i in range(0, len(numbers), size): # step by chunk
    size
    chunks.append(numbers[i:i + size])
print("Chunks:", chunks)
```

OUTPUT

```
Chunks: [[1, 2, 3], [4, 5, 6], [7]]
```

Concept: `range()` with a step plus slicing.

Problem 136. Insert into a Sorted List

Insert a value into a sorted list so the list stays sorted, without re-sorting the whole list.

SOLUTION

```
numbers = [2, 5, 8, 12]
value = 7
position = len(numbers) # default: goes at the end
for i in range(len(numbers)):
    if numbers[i] > value:
        position = i
        break
numbers.insert(position, value)
print("After insert:", numbers)
```

OUTPUT

```
After insert: [2, 5, 7, 8, 12]
```

Concept: finding the insertion point, then `list.insert()`.

Problem 137. Pairs with a Given Sum

Find all pairs of elements in a list that add up to a target value. Each pair should be reported once.

SOLUTION

```
numbers = [1, 2, 3, 4, 5, 6]
target = 7
for i in range(len(numbers)):
    for j in range(i + 1, len(numbers)): # j starts after
        i to avoid repeats
        if numbers[i] + numbers[j] == target:
            print("Pair:", numbers[i], "+", numbers[j],
                  "=", target)
```

OUTPUT

```
Pair: 1 + 6 = 7
Pair: 2 + 5 = 7
Pair: 3 + 4 = 7
```

Concept: nested loops over unique index pairs.

Problem 138. Merge Two Sorted Lists

Merge two already-sorted lists into one sorted list without using `sort()` or `sorted()`.

SOLUTION

```
a = [1, 3, 5]
b = [2, 4, 6, 8]
merged = []
i = 0
j = 0
while i < len(a) and j < len(b):
    if a[i] ≤ b[j]: # take the smaller front element
        merged.append(a[i])
        i += 1
    else:
        merged.append(b[j])
        j += 1
merged.extend(a[i:]) # one of these leftovers is empty
merged.extend(b[j:])
print("Merged:", merged)
```

OUTPUT

```
Merged: [1, 2, 3, 4, 5, 6, 8]
```

Concept: two-pointer merge, the core step of merge sort.

Problem 139. Flatten a Nested List

Flatten a list that may contain lists inside lists, at any depth, into a single flat list.

SOLUTION

```
def flatten(items):
    flat = []
    for item in items:
        if isinstance(item, list):
            flat.extend(flatten(item)) # recurse into
inner lists
        else:
            flat.append(item)
    return flat

nested = [1, [2, 3], [4, [5, 6]], 7]
print("Flattened:", flatten(nested))
```

OUTPUT

```
Flattened: [1, 2, 3, 4, 5, 6, 7]
```

Concept: recursion with `isinstance()` to handle any nesting depth.

Problem 140. Sort Without sort() (Bubble Sort)

Sort a list in ascending order without using `sort()` or `sorted()`, using the bubble sort algorithm.

SOLUTION

```
numbers = [64, 34, 25, 12, 22, 11, 90]
n = len(numbers)
for i in range(n - 1):
    for j in range(n - 1 - i): # the last i elements are
        already in place
        if numbers[j] > numbers[j + 1]:
            numbers[j], numbers[j + 1] = numbers[j + 1],
            numbers[j]
print("Sorted:", numbers)
```

OUTPUT

```
Sorted: [11, 12, 22, 25, 34, 64, 90]
```

Concept: bubble sort — repeatedly swapping adjacent out-of-order pairs.

Tuples & Sets

(Problems 141–152)

Problem 141. Tuple Creation & Unpacking

Create a tuple of two coordinates, access an element by index, and unpack the tuple into separate variables.

SOLUTION

```
point = (3, 4)
print(point[0])           # indexing works like a list

x, y = point              # unpacking: one variable per
element                  element
print(x, y)

colors = "red", "green", "blue"  # parentheses are
optional
print(colors)
```

OUTPUT

```
3
3 4
('red', 'green', 'blue')
```

Concept: tuple literals, indexing, and tuple unpacking.

Section 8 · Tuples & Sets (continued)

Problem 142. Swap Two Variables via Tuple Packing

Swap the values of two variables without using a temporary variable.

SOLUTION

```
a, b = 5, 10
print("Before:", a, b)

a, b = b, a    # right side is packed into a tuple, then
               unpacked
print("After:", a, b)
```

OUTPUT

```
Before: 5 10
After: 10 5
```

Concept: simultaneous assignment using tuple packing/unpacking.

Section 8 · Tuples & Sets (continued)

Problem 143. Tuple vs List Immutability

Show that a tuple cannot be modified after creation, while a list can. Catch the error instead of crashing.

SOLUTION

```
nums_tuple = (1, 2, 3)
nums_list = [1, 2, 3]

try:
    nums_tuple[0] = 99      # tuples are immutable
except TypeError as e:
    print("Tuple error:", e)

nums_list[0] = 99          # lists are mutable
print("List after change:", nums_list)
```

OUTPUT

```
Tuple error: 'tuple' object does not support item
assignment
List after change: [99, 2, 3]
```

Concept: immutability of tuples and try/except for TypeError.

Section 8 · Tuples & Sets (continued)

Problem 144. `count()` and `index()` on Tuples

Given a tuple of numbers, count how many times a value appears and find the position of another value.

SOLUTION

```
nums = (1, 2, 2, 3, 2, 4)

print("2 appears", nums.count(2), "times")
print("First 3 is at index", nums.index(3))
```

OUTPUT

```
2 appears 3 times
First 3 is at index 3
```

Concept: the only two tuple methods — `count()` and `index()`.

Section 8 · Tuples & Sets (continued)

Problem 145. Convert List to Tuple and Back

Convert a list to a tuple, then convert a tuple to a list so it can be sorted.

SOLUTION

```
fruits = ["mango", "apple", "banana"]
frozen = tuple(fruits)           # list → tuple
print(frozen)

nums = (5, 1, 4, 2)
as_list = list(nums)            # tuple → list (now
                                # editable)
as_list.sort()
print(as_list)
```

OUTPUT

```
('mango', 'apple', 'banana')
[1, 2, 4, 5]
```

Concept: type conversion with `tuple()` and `list()`.

Section 8 · Tuples & Sets (continued)

Problem 146. Min/Max in a Tuple of Tuples

Given a tuple of (name, score) pairs, find the pair with the highest and the lowest score.

SOLUTION

```
scores = ("Amit", 82), ("BeLa", 91), ("Chad", 77))

top = max(scores, key=lambda s: s[1])    # compare by
score, not name
low = min(scores, key=lambda s: s[1])

print("Highest:", top)
print("Lowest:", low)
```

OUTPUT

```
Highest: ('BeLa', 91)
Lowest: ('Chad', 77)
```

Concept: max()/min() with a key function on nested tuples.

Section 8 · Tuples & Sets (continued)

Problem 147. Set Union, Intersection & Difference

Given two sets of numbers, print their union, intersection, and difference.

SOLUTION

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print("Union:", sorted(a | b))          # or a.union(b)
print("Intersection:", sorted(a & b))    # or
a.intersection(b)
print("Difference a-b:", sorted(a - b))  # in a but not in
b
```

OUTPUT

```
Union: [1, 2, 3, 4, 5, 6]
Intersection: [3, 4]
Difference a-b: [1, 2]
```

Concept: basic set algebra with `|`, `&`, and `-` operators.

Section 8 · Tuples & Sets (continued)

Problem 148. Symmetric Difference

Find the elements that are in exactly one of two sets (not in both).

SOLUTION

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(sorted(a ^ b))           # operator form
print(sorted(a.symmetric_difference(b))) # method form
```

OUTPUT

```
[1, 2, 5, 6]
[1, 2, 5, 6]
```

Concept: symmetric difference = union minus intersection.

Section 8 · Tuples & Sets (continued)

Problem 149. Remove Duplicates Preserving Order

Remove duplicates from a list while keeping the original order of first appearance (plain `set()` would lose the order).

SOLUTION

```
def dedupe(items):  
    seen = set()           # fast membership checks  
    result = []  
    for item in items:  
        if item not in seen:  
            seen.add(item)  
            result.append(item)  
    return result  
  
print(dedupe([3, 1, 3, 2, 1, 5]))
```

OUTPUT

```
[3, 1, 2, 5]
```

Concept: using a set for $O(1)$ lookups while a list preserves order.

Section 8 · Tuples & Sets (continued)

Problem 150. Subset and Superset Checks

Check whether one set is a subset of another, and whether the second is a superset of the first.

SOLUTION

```
small = {1, 2}
big = {1, 2, 3, 4}

print(small.issubset(big))      # every element of small is
in big
print(big.issuperset(small))
print(small ≤ big)              # operator shortcut for
issubset
print(big < big)                # proper subset: must be
strictly smaller
```

OUTPUT

```
True
True
True
False
```

Concept: `issubset()/issuperset()` and the `<=` / `<` set operators.

Section 8 · Tuples & Sets (continued)

Problem 151. Unique Words in a Sentence

Given a sentence, print the unique words in alphabetical order and how many unique words there are.

SOLUTION

```
sentence = "the cat and the dog chased the cat"

words = set(sentence.lower().split())    # set drops
repeats
print("Unique words:", sorted(words))
print("Count:", len(words))
```

OUTPUT

```
Unique words: ['and', 'cat', 'chased', 'dog', 'the']
Count: 5
```

Concept: `split()` plus `set()` to deduplicate words.

Section 8 · Tuples & Sets (continued)

Problem 152. Frozen Set Demo

Create a frozenset, show that it cannot be modified, and use it as a dictionary key (a normal set cannot be a key).

SOLUTION

```
fs = frozenset([1, 2, 3])
print(fs)

try:
    fs.add(4)                # frozenset has no add/remove
except AttributeError as e:
    print("Error:", e)

# frozensets are hashable, so they can be dict keys
routes = {frozenset(["Delhi", "Mumbai"]): 1400}
print(routes[frozenset(["Mumbai", "Delhi"])]) # order
does not matter
```

OUTPUT

```
frozenset({1, 2, 3})
Error: 'frozenset' object has no attribute 'add'
1400
```

Concept: frozenset — an immutable, hashable set.

Dictionaries

(Problems 153–172)

Problem 153. Create, Access & Update a Dictionary

Create a dictionary describing a person, read one value, change an existing value, and add a new key.

SOLUTION

```
person = {"name": "Asha", "age": 25}

print(person["name"])    # access by key

person["age"] = 26       # update existing key
person["city"] = "Pune"  # add new key
print(person)
```

OUTPUT

```
Asha
{'name': 'Asha', 'age': 26, 'city': 'Pune'}
```

Concept: dictionary creation, key access, update, and insertion.

Section 9 · Dictionaries (continued)

Problem 154. Iterate Keys, Values & Items

Loop over a dictionary three ways: keys only, values only, and key-value pairs together.

SOLUTION

```
marks = {"math": 90, "science": 85}

for subject in marks:           # loops over keys by
    print(subject)              default

for score in marks.values():
    print(score)

for subject, score in marks.items():
    print(f"{subject} = {score}")
```

OUTPUT

```
math
science
90
85
math = 90
science = 85
```

Concept: dict iteration with `keys()`, `values()`, and `items()`.

Section 9 · Dictionaries (continued)

Problem 155. Safe Key Lookup with .get()

Look up keys that may not exist without crashing, and provide a default value for missing keys.

SOLUTION

```
person = {"name": "Ravi", "age": 30}

print(person.get("name"))           # key exists
print(person.get("city"))           # missing → None,
no crash
print(person.get("city", "Not found")) # missing →
custom default

# person["city"] would raise KeyError instead
print("city" in person)             # membership test
```

OUTPUT

```
Ravi
None
Not found
False
```

Concept: .get() with a default vs KeyError from [] access.

Section 9 · Dictionaries (continued)

Problem 156. Sum of Dictionary Values

A shopping cart maps item names to prices. Compute the total bill and the number of items.

SOLUTION

```
cart = {"apple": 30, "milk": 55, "bread": 40}

print("Items:", len(cart))
print("Total:", sum(cart.values()))
```

OUTPUT

```
Items: 3
Total: 125
```

Concept: aggregating with `sum()` over `dict.values()`.

Section 9 · Dictionaries (continued)

Problem 157. Key with the Maximum Value

Given student scores in a dictionary, find which student scored the highest.

SOLUTION

```
scores = {"Amit": 82, "BeLa": 91, "Chad": 77}

best = max(scores, key=scores.get)    # compare keys by
their values
print(f"{best} scored the highest: {scores[best]}")
```

OUTPUT

```
BeLa scored the highest: 91
```

Concept: `max()` with `key=dict.get` to rank keys by value.

Section 9 · Dictionaries (continued)

Problem 158. Dict from Two Lists via zip()

Build a dictionary from a list of names and a matching list of ages.

SOLUTION

```
names = ["Asha", "Ravi", "Meena"]
ages = [25, 30, 28]

people = dict(zip(names, ages))    # zip pairs items
position by position
print(people)
```

OUTPUT

```
{'Asha': 25, 'Ravi': 30, 'Meena': 28}
```

Concept: zip() to pair parallel lists, dict() to convert pairs.

Section 9 · Dictionaries (continued)

Problem 159. Dictionary Comprehension

Build a dictionary mapping each number from 1 to 5 to its square using a dict comprehension.

SOLUTION

```
squares = {n: n * n for n in range(1, 6)}  
print(squares)  
  
# with a condition: only even keys  
even_squares = {n: n * n for n in range(1, 11) if n % 2 ==  
0}  
print(even_squares)
```

OUTPUT

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}  
{2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

Concept: dict comprehension {key: value for ...}.

Section 9 · Dictionaries (continued)

Problem 160. Character Count with a Dict

Count how many times each character appears in a word, using a plain dictionary.

SOLUTION

```
word = "banana"
counts = {}

for ch in word:
    counts[ch] = counts.get(ch, 0) + 1    # 0 if seen first
time
print(counts)
```

OUTPUT

```
{'b': 1, 'a': 3, 'n': 2}
```

Concept: the counting pattern with `.get(key, 0) + 1`.

Section 9 · Dictionaries (continued)

Problem 161. Word Frequency Counter

Count how many times each word appears in a sentence.

SOLUTION

```
text = "to be or not to be"
freq = {}

for word in text.split():
    freq[word] = freq.get(word, 0) + 1
print(freq)
```

OUTPUT

```
{'to': 2, 'be': 2, 'or': 1, 'not': 1}
```

Concept: `split()` plus the dict counting pattern.

Section 9 · Dictionaries (continued)

Problem 162. Invert a Dictionary

Swap the keys and values of a dictionary (assume all values are unique).

SOLUTION

```
capitals = {"India": "Delhi", "Japan": "Tokyo"}

inverted = {city: country for country, city in
capitals.items()}
print(inverted)
```

OUTPUT

```
{'Delhi': 'India', 'Tokyo': 'Japan'}
```

Concept: swapping key/value pairs with a dict comprehension.

Section 9 · Dictionaries (continued)

Problem 163. Merge Two Dictionaries

Merge default settings with user settings. When the same key exists in both, the user setting must win.

SOLUTION

```
defaults = {"theme": "light", "font": 12}
user = {"theme": "dark", "lang": "en"}

merged = {**defaults, **user}    # later dict wins on
clashes
print(merged)

merged2 = defaults | user        # same result, Python 3.9+
print(merged2)
```

OUTPUT

```
{'theme': 'dark', 'font': 12, 'lang': 'en'}
{'theme': 'dark', 'font': 12, 'lang': 'en'}
```

Concept: merging with `**` unpacking and the `|` operator; last key wins.

Section 9 · Dictionaries (continued)

Problem 164. Filter a Dict by Condition

From a dictionary of product prices, keep only the products cheaper than 100.

SOLUTION

```
prices = {"pen": 10, "book": 150, "bag": 500, "pencil": 5}

cheap = {name: price for name, price in prices.items() if
price < 100}
print(cheap)
```

OUTPUT

```
{'pen': 10, 'pencil': 5}
```

Concept: dict comprehension with an if filter.

Section 9 · Dictionaries (continued)

Problem 165. Sort a Dict by Value

Sort student scores from highest to lowest and print the sorted dictionary.

SOLUTION

```
scores = {"Amit": 82, "BeLa": 91, "Chad": 77}

ranked = dict(sorted(scores.items(),
                     key=lambda item: item[1],    # sort by
the value                                     reverse=True))

print(ranked)
```

OUTPUT

```
{'BeLa': 91, 'Amit': 82, 'Chad': 77}
```

Concept: `sorted()` on `items()` with `key=lambda`, rebuilt into a dict.

Section 9 · Dictionaries (continued)

Problem 166. Nested Dictionary Access

Read values from a dictionary of dictionaries, including a safe lookup for an employee who may not exist.

SOLUTION

```
company = {
    "emp1": {"name": "Riya", "salary": 50000},
    "emp2": {"name": "Karan", "salary": 60000},
}

print(company["emp1"]["name"])          # chain the keys
print(company["emp2"]["salary"])

# safe access: missing employee gives {} instead of
# crashing
print(company.get("emp3", {}).get("name", "Unknown"))
```

OUTPUT

```
Riya
60000
Unknown
```

Concept: chained key access and chained `.get()` for nested dicts.

Section 9 · Dictionaries (continued)

Problem 167. Student Marks Average

Each student has a list of marks. Print each student's average mark.

SOLUTION

```
marks = {"Amit": [80, 90, 70], "Bela": [95, 85, 90]}

for name, scores in marks.items():
    average = sum(scores) / len(scores)
    print(f"{name}: {average:.1f}")
```

OUTPUT

```
Amit: 80.0
Bela: 90.0
```

Concept: dict of lists, items() iteration, sum()/len() average.

Section 9 · Dictionaries (continued)

Problem 168. Phone Book: Add, Search, Delete

Implement a small phone book with functions to add a contact, search by name, and delete a contact.

SOLUTION

```
phone_book = {}

def add(name, number):
    phone_book[name] = number

def search(name):
    return phone_book.get(name, "Not found")

def delete(name):
    if name in phone_book:
        del phone_book[name]
        return "Deleted"
    return "Not found"

add("Asha", "98765")
add("Ravi", "91234")
print(search("Asha"))
print(delete("Ravi"))
print(search("Ravi"))
```

OUTPUT

```
98765
Deleted
Not found
```

Concept: CRUD operations on a dict (add, get, del).

Section 9 · Dictionaries (continued)

Problem 169. Group Items by Key

Group a list of words by their first letter into a dictionary of lists.

SOLUTION

```
words = ["apple", "banana", "avocado", "cherry",
         "blueberry"]
groups = {}

for word in words:
    first = word[0]
    groups.setdefault(first, []).append(word)    # create
list if missing
print(groups)
```

OUTPUT

```
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'],
 'c': ['cherry']}
```

Concept: grouping with `dict.setdefault(key, [])`.

Section 9 · Dictionaries (continued)

Problem 170. Count Items by Category

Given (item, category) pairs, count how many items fall in each category.

SOLUTION

```
items = [("apple", "fruit"), ("carrot", "vegetable"),
         ("banana", "fruit"), ("potato", "vegetable"),
         ("mango", "fruit")]

counts = {}
for name, category in items:
    counts[category] = counts.get(category, 0) + 1
print(counts)
```

OUTPUT

```
{'fruit': 3, 'vegetable': 2}
```

Concept: tuple unpacking in a loop plus the dict counting pattern.

Section 9 · Dictionaries (continued)

Problem 171. collections.Counter

Use Counter to count letters in a word and show the two most common letters.

SOLUTION

```
from collections import Counter

letters = Counter("mississippi")    # counts everything in
one line
print(letters)
print(letters.most_common(2))
print(letters["s"])                # missing keys return
0, not an error
```

OUTPUT

```
Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
[('i', 4), ('s', 4)]
4
```

Concept: collections.Counter and most_common().

Section 9 · Dictionaries (continued)

Problem 172. collections.defaultdict Grouping

Group student names by their grade using a defaultdict so missing keys are created automatically.

SOLUTION

```
from collections import defaultdict

students = [("A", "Amit"), ("B", "BeLa"), ("A", "Asha")]

groups = defaultdict(list)    # missing key → empty list,
                               # automatically
for grade, name in students:
    groups[grade].append(name)

print(dict(groups))
```

OUTPUT

```
{'A': ['Amit', 'Asha'], 'B': ['BeLa']}
```

Concept: defaultdict(list) removes the "key missing" check when grouping.

Functions, Lambda & Recursion

(Problems 173–197)

Problem 173. Simple Function with Return

Write a function that returns the square of a number, and call it a few times.

SOLUTION

```
def square(n):  
    return n * n    # return sends the result back to the  
    caller  
  
print(square(5))  
print(square(square(2)))    # results can be reused  
result = square(10)  
print(result)
```

OUTPUT

```
25  
16  
100
```

Concept: def, parameters, and return values.

Problem 174. Default Arguments

Write a greeting function where the greeting word is optional and defaults to "Hello".

SOLUTION

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Asha"))           # uses the default
print(greet("Ravi", "Hi"))    # overrides the default
```

OUTPUT

```
Hello, Asha!
Hi, Ravi!
```

Concept: default parameter values.

Problem 175. Keyword Arguments

Call a function using keyword arguments so the order of arguments does not matter.

SOLUTION

```
def describe_pet(name, animal):  
    print(f"{name} is a {animal}")  
  
describe_pet("Bruno", "dog")           # positional:  
order matters  
describe_pet(animal="cat", name="Misty") # keyword: any  
order works
```

OUTPUT

```
Bruno is a dog  
Misty is a cat
```

Concept: positional vs keyword arguments.

Problem 176. Function Returning Multiple Values

Write a function that returns both the smallest and largest number in a list.

SOLUTION

```
def min_max(nums):  
    return min(nums), max(nums)    # actually returns one  
    tuple  
  
low, high = min_max([4, 1, 9, 6])    # unpack the tuple  
print("Min:", low)  
print("Max:", high)  
print(min_max([4, 1, 9, 6]))        # without unpacking
```

OUTPUT

```
Min: 1  
Max: 9  
(1, 9)
```

Concept: returning a tuple and unpacking multiple return values.

Problem 177. *args — Sum of Any Number of Arguments

Write a function that accepts any number of numbers and returns their sum.

SOLUTION

```
def total(*args):          # args collects extra arguments
    into a tuple
    return sum(args)

print(total(1, 2, 3))
print(total(5, 10, 15, 20))
print(total())             # works even with nothing
```

OUTPUT

```
6
50
0
```

Concept: *args variadic parameters.

Problem 178. **kwargs Printer

Write a function that accepts any keyword arguments and prints them as "key: value" lines.

SOLUTION

```
def show_info(**kwargs):    # kwargs collects keyword args
    into a dict
    for key, value in kwargs.items():
        print(f"{key}: {value}")

show_info(name="Asha", age=25, city="Pune")
```

OUTPUT

```
name: Asha
age: 25
city: Pune
```

Concept: **kwargs collecting keyword arguments into a dict.

Problem 179. Docstring + help() Demo

Write a documented function, then read its documentation with `__doc__` and `help()`.

SOLUTION

```
def area(length, width):  
    """Return the area of a rectangle."""  
    return length * width  
  
print(area(4, 5))  
print(area.__doc__)    # the docstring is stored on the  
                        # function  
help(area)              # help() formats it nicely
```

OUTPUT

```
20  
Return the area of a rectangle.  
Help on function area in module __main__:  
  
area(length, width)  
    Return the area of a rectangle.
```

Concept: docstrings, `__doc__`, and `help()`.

Problem 180. Prime Check as a Function

Write a reusable function `is_prime(n)` that returns `True` if `n` is prime, and test it.

SOLUTION

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):    # check up to
sqrt(n)
        if n % i == 0:
            return False
    return True

print(is_prime(29))
print(is_prime(30))
print([n for n in range(2, 20) if is_prime(n)])
```

OUTPUT

```
True
False
[2, 3, 5, 7, 11, 13, 17, 19]
```

Concept: wrapping logic in a function so it can be reused and tested.

Section 10 · Functions, Lambda & Recursion (continued)

Problem 181. Lambda Basics

Rewrite small one-line functions as lambdas and call them.

SOLUTION

```
square = lambda x: x * x          # same as: def square(x):  
return x * x  
add = lambda a, b: a + b  
  
print(square(6))  
print(add(3, 4))  
print((lambda s: s.upper())("hello"))  # lambdas can be  
used inline
```

OUTPUT

```
36  
7  
HELLO
```

Concept: lambda — an anonymous one-expression function.

Section 10 · Functions, Lambda & Recursion (continued)

Problem 182. map() to Square a List

Use map() with a lambda to square every number in a list.

SOLUTION

```
nums = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x * x, nums))    # map is
lazy, list() runs it
print(squares)

names = ["asha", "ravi"]
print(list(map(str.upper, names)))    # any function works,
not just lambda
```

OUTPUT

```
[1, 4, 9, 16, 25]
['ASHA', 'RAVI']
```

Concept: map() applies a function to every element.

Problem 183. filter() for Even Numbers

Use filter() with a lambda to keep only the even numbers from a list.

SOLUTION

```
nums = [10, 15, 20, 25, 30]

evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)

odds = list(filter(lambda x: x % 2 != 0, nums))
print(odds)
```

OUTPUT

```
[10, 20, 30]
[15, 25]
```

Concept: filter() keeps elements where the function returns True.

Problem 184. reduce() for Product

Use `functools.reduce()` to multiply all numbers in a list into a single product.

SOLUTION

```
from functools import reduce

nums = [1, 2, 3, 4, 5]

# reduce folds the list: (((1*2)*3)*4)*5
product = reduce(lambda x, y: x * y, nums)
print(product)
```

OUTPUT

```
120
```

Concept: `reduce()` combines a sequence into one value.

Section 10 · Functions, Lambda & Recursion (continued)

Problem 185. sorted() with key=lambda

Sort a list of (name, marks) tuples by marks in descending order, and sort a list of words by length.

SOLUTION

```
students = [("Amit", 82), ("Bela", 91), ("Chad", 77)]

by_marks = sorted(students, key=lambda s: s[1],
reverse=True)
print(by_marks)

words = ["kiwi", "banana", "fig"]
print(sorted(words, key=len))
```

OUTPUT

```
[('Bela', 91), ('Amit', 82), ('Chad', 77)]
['fig', 'kiwi', 'banana']
```

Concept: custom sort order via the key parameter.

Problem 186. Recursion: Factorial

Compute $n!$ recursively: $n! = n \times (n-1)!$, with $0! = 1$.

SOLUTION

```
def factorial(n):  
    if n ≤ 1:                                # base case stops the  
        recursion                            recursion  
        return 1  
    return n * factorial(n - 1)              # recursive case  
  
print(factorial(5))  
print(factorial(0))
```

OUTPUT

```
120  
1
```

Concept: recursion — base case plus recursive case.

Section 10 · Functions, Lambda & Recursion (continued)

Problem 187. Recursion: Sum of Digits

Recursively add the digits of a number, e.g. $1234 \rightarrow 1+2+3+4 = 10$.

SOLUTION

```
def sum_digits(n):  
    if n < 10:                                # single digit:  
        return n                               base case  
    return n % 10 + sum_digits(n // 10) # last digit +  
    rest  
  
print(sum_digits(1234))  
print(sum_digits(9))
```

OUTPUT

```
10  
9
```

Concept: recursion with `% 10` and `// 10` to peel off digits.

Section 10 · Functions, Lambda & Recursion (continued)

Problem 188. Recursion: Power

Compute base^{exp} recursively without using the `**` operator.

SOLUTION

```
def power(base, exp):  
    if exp == 0:          # anything to the power 0 is 1  
        return 1  
    return base * power(base, exp - 1)  
  
print(power(2, 10))  
print(power(5, 3))
```

OUTPUT

```
1024  
125
```

Concept: recursion — repeated multiplication as `base * power(base, exp-1)`.

Problem 189. Recursion: Fibonacci

Write a recursive fibonacci function and print the first 8 Fibonacci numbers.

SOLUTION

```
def fib(n):  
    if n ≤ 1:                # fib(0)=0, fib(1)=1  
        return n  
    return fib(n - 1) + fib(n - 2)  
  
print([fib(i) for i in range(8)])
```

OUTPUT

```
[0, 1, 1, 2, 3, 5, 8, 13]
```

Concept: recursion with two recursive calls per step.

Section 10 · Functions, Lambda & Recursion (continued)

Problem 190. Reverse a String Recursively

Reverse a string using recursion instead of slicing with `[::-1]`.

SOLUTION

```
def reverse(s):
    if len(s) ≤ 1:                # empty or 1 char:
        already reversed
        return s
    return reverse(s[1:]) + s[0]  # reverse the rest,
    then first char

print(reverse("python"))
print(reverse("a"))
```

OUTPUT

```
nohtyp
a
```

Concept: recursion on a shrinking slice of the string.

Problem 191. Recursion: GCD (Euclid's Algorithm)

Find the greatest common divisor of two numbers using the recursive Euclidean algorithm.

SOLUTION

```
def gcd(a, b):  
    if b == 0:                # when remainder hits 0, a is  
        the GCD              the GCD  
        return a  
    return gcd(b, a % b)  
  
print(gcd(48, 18))  
print(gcd(100, 75))
```

OUTPUT

```
6  
25
```

Concept: Euclid's algorithm — $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$.

Problem 192. Recursion vs Iteration Comparison

Compute factorial(10) both recursively and with a loop, confirm they match, and note the trade-offs.

SOLUTION

```
def fact_recursive(n):  
    return 1 if n ≤ 1 else n * fact_recursive(n - 1)  
  
def fact_iterative(n):  
    result = 1  
    for i in range(2, n + 1):  
        result *= i  
    return result  
  
print(fact_recursive(10))  
print(fact_iterative(10))  
print(fact_recursive(10) == fact_iterative(10))  
# Recursion is shorter but each call uses stack memory  
# (deep inputs can raise RecursionError); loops use  
constant memory.
```

OUTPUT

```
3628800  
3628800  
True
```

Concept: same algorithm two ways — call-stack cost vs loop simplicity.

Section 10 · Functions, Lambda & Recursion (continued)

Problem 193. Tower of Hanoi

Print the moves needed to transfer 3 disks from rod A to rod C using rod B, moving one disk at a time and never placing a larger disk on a smaller one.

SOLUTION

```
def hanoi(n, source, target, helper):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    hanoi(n - 1, source, helper, target)  # move smaller pile aside
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n - 1, helper, target, source)  # bring it back on top

hanoi(3, "A", "C", "B")
print("Total moves:", 2 ** 3 - 1)
```

OUTPUT

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves: 7
```

Concept: classic divide-and-conquer recursion ($2^n - 1$ moves).

Problem 194. Closures: Counter Maker

Write `make_counter()` that returns a new counting function each time it is called. Each counter must remember its own count.

SOLUTION

```
def make_counter():
    count = 0
    def counter():
        nonlocal count    # modify the enclosing variable,
        # not a new one
        count += 1
        return count
    return counter        # the returned function
    "remembers" count

c1 = make_counter()
c2 = make_counter()
print(c1(), c1(), c1())  # c1 has its own count
print(c2())              # c2 starts fresh
```

OUTPUT

```
1 2 3
1
```

Concept: closures and the `nonlocal` keyword.

Section 10 · Functions, Lambda & Recursion (continued)

Problem 195. Generator Function with yield

Write a countdown generator that yields numbers from n down to 1, and consume it with next() and a loop.

SOLUTION

```
def countdown(n):
    while n > 0:
        yield n      # pause here and hand out one value
        n -= 1

gen = countdown(3)
print(next(gen))    # values come one at a time, on demand
print(next(gen))

for value in countdown(5):
    print(value, end=" ")
print()
```

OUTPUT

```
3
2
5 4 3 2 1
```

Concept: yield turns a function into a lazy generator.

Section 10 · Functions, Lambda & Recursion (continued)

Problem 196. Generator for Even Numbers

Write a generator that yields even numbers up to a limit, then use it to list evens up to 10 and sum evens up to 100.

SOLUTION

```
def evens(limit):
    n = 2
    while n ≤ limit:
        yield n
        n += 2

print(list(evens(10)))
print(sum(evens(100)))    # generators feed directly into
sum()
# Unlike a list, the generator never stores all numbers in
memory.
```

OUTPUT

```
[2, 4, 6, 8, 10]
2550
```

Concept: generators produce values lazily with constant memory.

Problem 197. Simple Logging Decorator

Write a decorator that logs a function's name and arguments before it runs and its result after, then apply it with @.

SOLUTION

```
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with {args}")
        result = func(*args, **kwargs)  # run the real
function
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper

@logger      # same as: add = logger(add)
def add(a, b):
    return a + b

print(add(3, 4))
```

OUTPUT

```
Calling add with (3, 4)
add returned 7
7
```

Concept: decorators — functions that wrap other functions;
*args/**kwargs pass-through.

Comprehensions & Pythonic Idioms

(Problems 198–209)

Problem 198. List Comprehension: Squares and Evens

Using list comprehensions, build a list of the squares of the numbers 1 to 10, and a second list containing only the even numbers from 1 to 10.

SOLUTION

```
squares = [n ** 2 for n in range(1, 11)]          #  
transform every item  
evens = [n for n in range(1, 11) if n % 2 == 0]  # keep  
only items passing the filter  
  
print("Squares:", squares)  
print("Evens:  ", evens)
```

OUTPUT

```
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]  
Evens:   [2, 4, 6, 8, 10]
```

Concept: list comprehension with a transform expression and an optional if filter.

Problem 199. Conditional Expression (Ternary)

Read a number and print "even" or "odd" using a single conditional expression (Python's ternary operator) instead of a full if/else block.

SOLUTION

```
n = int(input("Enter a number: "))

# value_if_true if condition else value_if_false
label = "even" if n % 2 == 0 else "odd"
print(f"{n} is {label}")
```

OUTPUT

```
Enter a number: 7
7 is odd
```

Concept: conditional (ternary) expression — an if/else that produces a value on one line.

Problem 200. Comprehension with if-else

Given a list of numbers, build a new list where every negative number is replaced by 0 and every non-negative number is kept as is, using one list comprehension.

SOLUTION

```
numbers = [4, -2, 7, -9, 0, 3, -1]

# if-else goes BEFORE the for when it chooses the value
cleaned = [n if n ≥ 0 else 0 for n in numbers]
print("Original:", numbers)
print("Cleaned: ", cleaned)
```

OUTPUT

```
Original: [4, -2, 7, -9, 0, 3, -1]
Cleaned:  [4, 0, 7, 0, 0, 3, 0]
```

Concept: conditional expression inside a list comprehension (value position, not filter position).

Problem 201. Dict Comprehension

Given a list of words, build a dictionary mapping each word to its length using a dict comprehension.

SOLUTION

```
words = ["python", "is", "fun", "to", "learn"]

lengths = {word: len(word) for word in words}
print(lengths)

# with a filter: only words longer than 2 letters
long_words = {word: len(word) for word in words if
len(word) > 2}
print(long_words)
```

OUTPUT

```
{'python': 6, 'is': 2, 'fun': 3, 'to': 2, 'learn': 5}
{'python': 6, 'fun': 3, 'learn': 5}
```

Concept: dict comprehension — {key: value for item in iterable}.

Problem 202. Set Comprehension

Given a list of words with mixed capitalisation, build a set of the unique first letters (lower-cased) using a set comprehension.

SOLUTION

```
words = ["Apple", "banana", "Avocado", "cherry",
         "Blueberry"]

# a set automatically removes duplicates
first_letters = {word[0].lower() for word in words}
print(first_letters)
print("Count of unique first letters:",
      len(first_letters))
```

OUTPUT

```
{'a', 'b', 'c'}
Count of unique first letters: 3
```

Concept: set comprehension — comprehension with curly braces that deduplicates automatically.

Problem 203. Zip Two Lists into Pairs

Given a list of names and a list of scores, pair them together with `zip()`. Print each pair, then build a dictionary from the two lists.

SOLUTION

```
names = ["Asha", "Ravi", "Meera"]
scores = [85, 92, 78]

for name, score in zip(names, scores): # zip pairs items
    by position
    print(f"{name}: {score}")

score_map = dict(zip(names, scores)) # pairs convert
directly to a dict
print(score_map)
```

OUTPUT

```
Asha: 85
Ravi: 92
Meera: 78
{'Asha': 85, 'Ravi': 92, 'Meera': 78}
```

Concept: `zip()` to iterate two sequences in parallel and `dict(zip(...))` to combine them.

Section 11 · Comprehensions & Pythonic Idioms (continued)

Problem 204. Enumerate with Index

Print a numbered menu from a list of dishes, with numbering starting at 1, using `enumerate()` instead of a manual counter.

SOLUTION

```
dishes = ["Dosa", "Paneer Tikka", "Biryani", "Gulab Jamun"]

for i, dish in enumerate(dishes, start=1): # enumerate
    yields (index, item)
    print(f"{i}. {dish}")
```

OUTPUT

```
1. Dosa
2. Paneer Tikka
3. Biryani
4. Gulab Jamun
```

Concept: `enumerate()` gives index and value together, avoiding a manual counter variable.

Section 11 · Comprehensions & Pythonic Idioms (continued)

Problem 205. any() and all() Checks

Given a list of exam marks, use all() to check whether every student passed (mark $\geq$ 40) and any() to check whether anyone scored a distinction (mark $\geq$ 90).

SOLUTION

```
marks = [55, 78, 42, 91, 66]

everyone_passed = all(m  $\geq$  40 for m in marks) # True only
if ALL are True
has_distinction = any(m  $\geq$  90 for m in marks) # True if
AT LEAST ONE is True

print("Everyone passed: ", everyone_passed)
print("Has a distinction:", has_distinction)
```

OUTPUT

```
Everyone passed:  True
Has a distinction: True
```

Concept: any()/all() with generator expressions for whole-collection boolean checks.

Problem 206. `sorted()` with a key Function

Sort a list of words by their length, and sort a list of (name, marks) tuples by marks in descending order, using the key argument of `sorted()`.

SOLUTION

```
words = ["kiwi", "banana", "fig", "apple"]
print(sorted(words, key=len)) # key tells sorted WHAT to
                              # compare

students = [("Asha", 85), ("Ravi", 92), ("Meera", 78)]
by_marks = sorted(students, key=lambda s: s[1],
                  reverse=True)
print(by_marks)
```

OUTPUT

```
['fig', 'kiwi', 'apple', 'banana']
[('Ravi', 92), ('Asha', 85), ('Meera', 78)]
```

Concept: `sorted()` with `key=` and `lambda` to sort by a computed value; `reverse=True` for descending.

Problem 207. Unpacking with * (first, *rest)

Given a list of numbers, use starred unpacking to split it into the first element and the rest, and separately into the beginning and the last element.

SOLUTION

```
numbers = [10, 20, 30, 40, 50]

first, *rest = numbers      # * collects the leftovers into
a list
print("first:", first)
print("rest: ", rest)

*init, last = numbers
print("init: ", init)
print("last: ", last)
```

OUTPUT

```
first: 10
rest:  [20, 30, 40, 50]
init:  [10, 20, 30, 40]
last:  50
```

Concept: extended iterable unpacking with the * operator.

Problem 208. Nested Comprehension: Flatten a Matrix

Given a 2D list (matrix), flatten it into a single list using a nested list comprehension.

SOLUTION

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

# read the fors left to right, exactly like nested loops
flat = [value for row in matrix for value in row]
print(flat)
```

OUTPUT

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Concept: nested list comprehension — two for clauses reading outer-to-inner.

Problem 209. Swap Min and Max in a List (One-Liner)

Given a list of distinct numbers, swap the positions of its smallest and largest elements using Python's tuple-swap idiom in a single statement.

SOLUTION

```
nums = [3, 9, 1, 7, 5]
print("Before:", nums)

i, j = nums.index(min(nums)), nums.index(max(nums))
nums[i], nums[j] = nums[j], nums[i] # tuple swap: no temp
variable needed

print("After: ", nums)
```

OUTPUT

```
Before: [3, 9, 1, 7, 5]
After:  [3, 1, 9, 7, 5]
```

Concept: tuple unpacking for in-place swap, combined with `min()/max()/index()`.

Object-Oriented Programming

(Problems 210–231)

Problem 210. Simple Class and Object

Define an empty-ish Dog class with one method bark() that prints "Woof!". Create an object from the class and call its method.

SOLUTION

```
class Dog:
    def bark(self):          # self = the object the method
                             # is called on
        print("Woof!")

my_dog = Dog()              # create an object (instance)
                             # of the class
my_dog.bark()
print(type(my_dog))
```

OUTPUT

```
Woof!
<class '__main__.Dog'>
```

Concept: class definition, instantiation, and calling an instance method.

Section 12 · Object-Oriented Programming (continued)

Problem 211. The `__init__` Constructor

Create a `Person` class whose constructor accepts a name and an age and stores them on the object. Create two people and print their details.

SOLUTION

```
class Person:
    def __init__(self, name, age): # runs automatically
    when object is created
        self.name = name          # attach data to THIS
    object
        self.age = age

p1 = Person("Asha", 25)
p2 = Person("Ravi", 30)
print(p1.name, "is", p1.age)
print(p2.name, "is", p2.age)
```

OUTPUT

```
Asha is 25
Ravi is 30
```

Concept: `__init__` constructor and instance attributes via `self`.

Section 12 · Object-Oriented Programming (continued)

Problem 212. Instance vs Class Variables

Create a Dog class with a class variable species shared by all dogs and an instance variable name unique to each dog. Show that species is shared while name is not.

SOLUTION

```
class Dog:
    species = "Canis familiaris"    # class variable:
    shared by every Dog

    def __init__(self, name):
        self.name = name           # instance variable:
    unique per object

a = Dog("Rex")
b = Dog("Buddy")
print(a.name, "-", a.species)
print(b.name, "-", b.species)
print("Same species object:", a.species is b.species)
```

OUTPUT

```
Rex - Canis familiaris
Buddy - Canis familiaris
Same species object: True
```

Concept: class variables (shared) vs instance variables (per-object).

Section 12 · Object-Oriented Programming (continued)

Problem 213. Instance Methods

Create a Rectangle class with width and height, plus methods area() and perimeter() that use the object's own data.

SOLUTION

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

r = Rectangle(4, 6)
print("Area:      ", r.area())
print("Perimeter:", r.perimeter())
```

OUTPUT

```
Area:      24
Perimeter: 20
```

Concept: instance methods operating on the object's own attributes.

Section 12 · Object-Oriented Programming (continued)

Problem 214. `__str__` and `__repr__`

Create a `Book` class that defines both `__str__` (friendly, for users) and `__repr__` (unambiguous, for developers). Show how `print()` and the raw object display differ.

SOLUTION

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        # used by print()
        return f"'{self.title}' by {self.author}"

    def __repr__(self):
        # used in debugging / containers
        return f"Book(title={self.title!r}, author={self.author!r})"

b = Book("Wings of Fire", "A.P.J. Abdul Kalam")
print(b)           # calls __str__
print(repr(b))     # calls __repr__
print([b])         # lists show items using __repr__
```

OUTPUT

```
'Wings of Fire' by A.P.J. Abdul Kalam
Book(title='Wings of Fire', author='A.P.J. Abdul Kalam')
[Book(title='Wings of Fire', author='A.P.J. Abdul Kalam')]
```

Concept: dunder methods `__str__` vs `__repr__` and when Python uses each.

Section 12 · Object-Oriented Programming (continued)

Problem 215. Inheritance

Create an `Animal` base class with `eat()`, and a `Cat` class that inherits from `Animal` and adds `meow()`. Show that a `Cat` can use both methods.

SOLUTION

```
class Animal:
    def eat(self):
        print("This animal is eating.")

class Cat(Animal):          # Cat inherits everything
    from Animal
    def meow(self):
        print("Meow!")

c = Cat()
c.eat()      # inherited from Animal
c.meow()     # defined on Cat
print(isinstance(c, Animal))
```

OUTPUT

```
This animal is eating.
Meow!
True
```

Concept: single inheritance — a child class reuses the parent's methods.

Section 12 • Object-Oriented Programming (continued)

Problem 216. Method Overriding

Create an `Animal` class with `speak()` that prints a generic message, then override `speak()` in `Dog` and `Cat` subclasses. Loop over a mixed list and call `speak()` on each.

SOLUTION

```
class Animal:
    def speak(self):
        print("Some generic sound")

class Dog(Animal):
    def speak(self):           # same name replaces the
parent's version              parent's version
    print("Woof!")

class Cat(Animal):
    def speak(self):
        print("Meow!")

for animal in [Dog(), Cat(), Animal()]:
    animal.speak()             # each object uses ITS OWN
speak()
```

OUTPUT

```
Woof!
Meow!
Some generic sound
```

Concept: method overriding and polymorphism — the object's actual class decides which method runs.

Section 12 • Object-Oriented Programming (continued)

Problem 217. Using super()

Create a Vehicle class storing a brand, and a Car subclass that adds a model. Use `super().__init__()` so the parent constructor still runs.

SOLUTION

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand)    # let the parent set up
        brand                      # then add the child's
        self.model = model         own data

c = Car("Tata", "Nexon")
print(c.brand, c.model)
```

OUTPUT

```
Tata Nexon
```

Concept: `super()` to call the parent class's constructor from a subclass.

Section 12 • Object-Oriented Programming (continued)

Problem 218. Encapsulation with `_` and `@property`

Create a `Temperature` class that stores celsius in a "private" attribute `_celsius`. Expose it through a `@property` getter and a setter that rejects values below absolute zero (-273.15).

SOLUTION

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius          # _ means "internal, please don't touch"

    @property
    def celsius(self):
        return self._celsius            # read like an attribute: t.celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self._celsius = value

t = Temperature(25)
print(t.celsius)
t.celsius = 30
print(t.celsius)
try:
    t.celsius = -300
except ValueError as e:
    print("Error:", e)
```

OUTPUT

```
25
30
Error: Below absolute zero!
```

Concept: encapsulation — underscore convention plus `@property` getter/setter with validation.

Section 12 · Object-Oriented Programming (continued)

Problem 219. @classmethod Alternative Constructor

Create a Person class with name and age, plus a class method from_birth_year(name, year) that computes the age and returns a new Person (assume the current year is 2026).

SOLUTION

```
class Person:
    CURRENT_YEAR = 2026

    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_birth_year(cls, name, year): # cls is the
class itself
        return cls(name, cls.CURRENT_YEAR - year)

p = Person.from_birth_year("Asha", 2000)
print(p.name, "is", p.age)
```

OUTPUT

```
Asha is 26
```

Concept: @classmethod as an alternative constructor using cls.

Section 12 • Object-Oriented Programming (continued)

Problem 220. @staticmethod Utility

Create a MathUtils class with static methods `is_even(n)` and `celsius_to_fahrenheit(c)` that need no object or class data, and call them directly on the class.

SOLUTION

```
class MathUtils:
    @staticmethod
    def is_even(n):                                # no self, no cls:
just a grouped function
    return n % 2 == 0

    @staticmethod
    def celsius_to_fahrenheit(c):
        return c * 9 / 5 + 32

print(MathUtils.is_even(10))
print(MathUtils.celsius_to_fahrenheit(37))
```

OUTPUT

```
True
98.6
```

Concept: `@staticmethod` — namespacing a plain function inside a class.

Section 12 • Object-Oriented Programming (continued)

Problem 221. Counting Instances with a Class Variable

Create a Robot class that counts how many robots have been created, using a class variable that every `__init__` call increments.

SOLUTION

```
class Robot:
    count = 0                # lives on the class,
                             shared by all

    def __init__(self, name):
        self.name = name
        Robot.count += 1     # update the SHARED
                             counter

r1 = Robot("R2")
r2 = Robot("C3P0")
r3 = Robot("Wall-E")
print("Robots created:", Robot.count)
```

OUTPUT

```
Robots created: 3
```

Concept: class variable as a shared counter across all instances.

Section 12 · Object-Oriented Programming (continued)

Problem 222. Student Class with Average

Create a Student class that stores a name and a list of grades. Add methods `add_grade(g)` and `average()`. Print the student's average after adding a few grades.

SOLUTION

```
class Student:
    def __init__(self, name):
        self.name = name
        self.grades = []          # each student gets
their OWN list

    def add_grade(self, grade):
        self.grades.append(grade)

    def average(self):
        if not self.grades:      # avoid dividing by zero
            return 0
        return sum(self.grades) / len(self.grades)

s = Student("Meera")
for g in [85, 90, 78]:
    s.add_grade(g)
print(f"{s.name}'s average: {s.average():.2f}")
```

OUTPUT

```
Meera's average: 84.33
```

Concept: a class managing its own mutable state (list of grades) through methods.

Section 12 • Object-Oriented Programming (continued)

Problem 223. Bank Account Class

Create a `BankAccount` class with `deposit(amount)` and `withdraw(amount)`. Withdrawals that exceed the balance must be rejected with a message. Demonstrate a few transactions.

SOLUTION

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. Balance: {self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print(f"Insufficient funds! Balance is {self.balance}")
        else:
            self.balance -= amount
            print(f"Withdrew {amount}. Balance: {self.balance}")

acc = BankAccount("Ravi", 100)
acc.deposit(50)
acc.withdraw(30)
acc.withdraw(500)
```

OUTPUT

```
Deposited 50. Balance: 150
Withdrew 30. Balance: 120
Insufficient funds! Balance is 120
```

Concept: methods that guard and mutate internal state (a classic encapsulation exercise).

Section 12 • Object-Oriented Programming (continued)

Problem 224. Operator Overloading (`__add__`, `__eq__`, `__lt__`)

Create a Money class so that two Money objects can be added with `+`, compared with `==` and `<`, and sorted. Implement `__add__`, `__eq__` and `__lt__`.

SOLUTION

```
class Money:
    def __init__(self, rupees):
        self.rupees = rupees

    def __add__(self, other):          # runs for: a + b
        return Money(self.rupees + other.rupees)

    def __eq__(self, other):          # runs for: a == b
        return self.rupees == other.rupees

    def __lt__(self, other):          # runs for: a < b (also enables
sorted)
        return self.rupees < other.rupees

    def __repr__(self):
        return f"Money({self.rupees})"

a, b = Money(100), Money(250)
print(a + b)
print(a == Money(100))
print(a < b)
print(sorted([b, a, Money(50)]))
```

OUTPUT

```
Money(350)
True
True
[Money(50), Money(100), Money(250)]
```

Concept: operator overloading with dunder methods `__add__`, `__eq__`, `__lt__`.

Section 12 · Object-Oriented Programming (continued)

Problem 225. `__len__` on a Custom Class

Create a Playlist class holding a list of songs, and implement `__len__` so the built-in `len()` works on a Playlist object.

SOLUTION

```
class Playlist:
    def __init__(self, name):
        self.name = name
        self.songs = []

    def add(self, song):
        self.songs.append(song)

    def __len__(self):                # makes len(playlist)
work                                work
        return len(self.songs)

p = Playlist("Road Trip")
p.add("Song A")
p.add("Song B")
p.add("Song C")
print(f"'{p.name}' has {len(p)} songs")
```

OUTPUT

```
'Road Trip' has 3 songs
```

Concept: `__len__` dunder method hooks a class into the built-in `len()`.

Section 12 • Object-Oriented Programming (continued)

Problem 226. Multiple Inheritance and MRO

Create classes A, B(A), C(A) each defining who(), and D(B, C) defining nothing. Show which who() D uses and print D's Method Resolution Order (MRO).

SOLUTION

```
class A:
    def who(self):
        return "A"

class B(A):
    def who(self):
        return "B"

class C(A):
    def who(self):
        return "C"

class D(B, C):      # inherits from BOTH B and C
    pass

d = D()
print("D().who() returns:", d.who())    # B wins: it is
first in D(B, C)
print([cls.__name__ for cls in D.__mro__])
```

OUTPUT

```
D().who() returns: B
['D', 'B', 'C', 'A', 'object']
```

Concept: multiple inheritance and the Method Resolution Order (C3 linearisation).

Section 12 • Object-Oriented Programming (continued)

Problem 227. Abstract Base Class with abc

Using the abc module, create an abstract Shape class with an abstract method area(). Implement Circle and Square subclasses and show that Shape itself cannot be instantiated.

SOLUTION

```
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self):          # subclasses MUST implement this
        pass

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return math.pi * self.r ** 2

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2

for shape in [Circle(2), Square(3)]:
    print(f"{type(shape).__name__} area: {shape.area():.2f}")

try:
    Shape()
except TypeError as e:
    print("Error:", e)  # abstract class: creating it is an error
```

OUTPUT

```
Circle area: 12.57
Square area: 9.00
Error: Can't instantiate abstract class Shape without an implementation for
abstract method 'area'
```

Concept: abstract base classes (ABC) and @abstractmethod enforcing an interface.

Section 12 · Object-Oriented Programming (continued)

Problem 228. Composition: Car has an Engine

Model a Car that contains an Engine object (composition, a "has-a" relationship). Calling `car.start()` should delegate to its engine.

SOLUTION

```
class Engine:
    def __init__(self, horsepower):
        self.horsepower = horsepower

    def start(self):
        print(f"Engine with {self.horsepower} HP starting... vroom!")

class Car:
    def __init__(self, brand, horsepower):
        self.brand = brand
        self.engine = Engine(horsepower)    # Car HAS an Engine

    def start(self):
        print(f"{self.brand}:")
        self.engine.start()                # delegate the work

car = Car("Mahindra", 150)
car.start()
```

OUTPUT

```
Mahindra:
Engine with 150 HP starting... vroom!
```

Concept: composition ("has-a") and delegation, versus inheritance ("is-a").

Section 12 • Object-Oriented Programming (continued)

Problem 229. Employee / Manager Hierarchy

Create an Employee class with name, salary and an info() method. Create a Manager subclass that also keeps a list of team members and overrides info() to include the team size, reusing the parent's info() via super().

SOLUTION

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def info(self):
        return f"{self.name}, salary {self.salary}"

class Manager(Employee):
    def __init__(self, name, salary):
        super().__init__(name, salary)
        self.team = []

    def add_report(self, employee):
        self.team.append(employee)

    def info(self):
        # extend, don't rewrite
        return super().info() + f", manages {len(self.team)} people"

e1 = Employee("Asha", 50000)
e2 = Employee("Ravi", 55000)
m = Manager("Meera", 90000)
m.add_report(e1)
m.add_report(e2)
print(e1.info())
print(m.info())
```

OUTPUT

```
Asha, salary 50000
Meera, salary 90000, manages 2 people
```

Concept: inheritance hierarchy with overriding that extends the parent via super().

Section 12 • Object-Oriented Programming (continued)

Problem 230. Mini Library Book System

Build a small library: a Book class with a title and an available flag, and a Library class with borrow(title) and give_back(title). Borrowing an unavailable book must fail politely.

SOLUTION

```
class Book:
    def __init__(self, title):
        self.title = title
        self.available = True

class Library:
    def __init__(self, titles):
        self.books = {t: Book(t) for t in titles}

    def borrow(self, title):
        book = self.books.get(title)
        if book is None:
            print(f"'{title}' is not in this library.")
        elif not book.available:
            print(f"'{title}' is already borrowed.")
        else:
            book.available = False
            print(f"You borrowed '{title}'.")

    def give_back(self, title):
        self.books[title].available = True
        print(f"You returned '{title}'.")

lib = Library(["Python 101", "Clean Code"])
lib.borrow("Python 101")
lib.borrow("Python 101")    # second borrow must fail
lib.give_back("Python 101")
lib.borrow("Python 101")
```

OUTPUT

```
You borrowed 'Python 101'.
'Python 101' is already borrowed.
You returned 'Python 101'.
You borrowed 'Python 101'.
```

Concept: two classes cooperating — objects stored in a dict, state flags, and defensive checks.

Section 12 • Object-Oriented Programming (continued)

Problem 231. Dataclass Demo

Use the `@dataclass` decorator to define a `Point` with `x` and `y`. Show that `__init__`, `__repr__` and `__eq__` are generated automatically, and use a default value for a label field.

SOLUTION

```
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int
    label: str = "origin-relative" # default value

p1 = Point(3, 4)                # __init__ written for us
p2 = Point(3, 4)
print(p1)                        # __repr__ written for us
print(p1 == p2)                 # __eq__ written for us
                                # (compares field values)
print(p1.label)
```

OUTPUT

```
Point(x=3, y=4, label='origin-relative')
True
origin-relative
```

Concept: `@dataclass` auto-generates `__init__`, `__repr__` and `__eq__` from typed fields.

File Handling

(Problems 232–243)

Problem 232. Write to a File

Create a file named notes.txt and write three lines of text into it using a with statement.

SOLUTION

```
with open("notes.txt", "w") as f:    # "w"
    creates/overwrites the file
    f.write("First line\n")
    f.write("Second line\n")
    f.write("Third line\n")
# the with block closes the file automatically
print("notes.txt written.")
```

OUTPUT

```
notes.txt written.

notes.txt now contains:
First line
Second line
Third line
```

Concept: open() in write mode and the with statement for automatic file closing.

Section 13 · File Handling (continued)

Problem 233. Read a Whole File

Open notes.txt (from the previous problem) and read its entire content into one string with read(), then print it.

SOLUTION

```
with open("notes.txt", "r") as f:    # "r" is read mode
    (the default)
    content = f.read()               # whole file as one
    string
    print(content)
```

OUTPUT

```
First line
Second line
Third line
```

Concept: read() loads the complete file content into a single string.

Section 13 · File Handling (continued)

Problem 234. Read a File Line by Line

Read notes.txt one line at a time by looping over the file object, and print each line with its line number.

SOLUTION

```
with open("notes.txt") as f:
    for number, line in enumerate(f, start=1): # a file
        iterates line by line
            print(number, line.strip())        # strip()
removes the trailing \n
```

OUTPUT

```
1 First line
2 Second line
3 Third line
```

Concept: iterating over a file object reads it lazily, one line at a time.

Section 13 · File Handling (continued)

Problem 235. Append to a File

Open notes.txt in append mode and add a fourth line without erasing the existing content. Print the file afterwards to confirm.

SOLUTION

```
with open("notes.txt", "a") as f:    # "a" adds to the end,
    never erases
    f.write("Fourth line (appended)\n")

with open("notes.txt") as f:
    print(f.read())
```

OUTPUT

```
First line
Second line
Third line
Fourth line (appended)
```

Concept: append mode "a" versus write mode "w" (which would overwrite).

Section 13 · File Handling (continued)

Problem 236. Count Lines, Words and Characters

Read notes.txt and print how many lines, words and characters it contains.

SOLUTION

```
with open("notes.txt") as f:
    text = f.read()

lines = text.splitlines()           # split on newlines
words = text.split()                # split on any
                                     # whitespace

print("Lines:      ", len(lines))
print("Words:      ", len(words))
print("Characters:", len(text))
```

OUTPUT

```
Lines:      4
Words:      10
Characters: 63
```

Concept: `splitlines()` and `split()` for counting units of text read from a file.

Section 13 · File Handling (continued)

Problem 237. Write a List to a File and Read It Back

Save a list of fruits to fruits.txt, one item per line, then read the file back into a new list and prove the lists are equal.

SOLUTION

```
fruits = ["apple", "banana", "cherry", "mango"]

with open("fruits.txt", "w") as f:
    f.write("\n".join(fruits))          # join list items
with open("fruits.txt") as f:
    loaded = f.read().splitlines()      # back to a list of
strings

print("Loaded:", loaded)
print("Round-trip OK:", loaded == fruits)
```

OUTPUT

```
Loaded: ['apple', 'banana', 'cherry', 'mango']
Round-trip OK: True
```

Concept: serialising a simple list as lines of text and parsing it back.

Section 13 · File Handling (continued)

Problem 238. Copy a File

Copy notes.txt into backup.txt by reading the source and writing the destination, then print the copy's content to verify.

SOLUTION

```
with open("notes.txt") as src:
    data = src.read()

with open("backup.txt", "w") as dst:
    dst.write(data)

with open("backup.txt") as f:
    print(f.read())
# tip: the shutil module does this in one call:
shutil.copyfile(src, dst)
```

OUTPUT

```
First line
Second line
Third line
Fourth line (appended)
```

Concept: file copy as read-then-write; shutil.copyfile as the standard-library shortcut.

Section 13 · File Handling (continued)

Problem 239. Check if a File Exists

Before opening a file, check whether it exists using both `os.path.exists()` and `pathlib.Path.exists()`. Test with one file that exists and one that does not.

SOLUTION

```
import os
from pathlib import Path

print(os.path.exists("notes.txt"))          # classic way
print(os.path.exists("missing.txt"))

p = Path("notes.txt")                       # modern,
object-oriented way
print(p.exists(), "-", p.name, "-", p.suffix)
```

OUTPUT

```
True
False
True - notes.txt - .txt
```

Concept: existence checks with `os.path` and the `pathlib.Path` object API.

Section 13 · File Handling (continued)

Problem 240. Find and Replace in a File

Replace every occurrence of the word "line" with "row" in notes.txt: read the file, transform the text, and write it back. Print the result.

SOLUTION

```
with open("notes.txt") as f:
    text = f.read()

text = text.replace("line", "row")    # strings are
immutable: this makes a new one

with open("notes.txt", "w") as f:    # overwrite with the
    edited version
    f.write(text)

with open("notes.txt") as f:
    print(f.read())
```

OUTPUT

```
First row
Second row
Third row
Fourth row (appended)
```

Concept: read-modify-write pattern with `str.replace()` for in-file substitution.

Section 13 · File Handling (continued)

Problem 241. Merge Two Files

Given part1.txt and part2.txt, create merged.txt containing the content of both, in order. Print the merged file.

SOLUTION

```
with open("part1.txt", "w") as f:
    f.write("Hello from part 1\n")
with open("part2.txt", "w") as f:
    f.write("Hello from part 2\n")

with open("merged.txt", "w") as out:
    for name in ["part1.txt", "part2.txt"]:
        with open(name) as src:
            out.write(src.read())    # append each source
in turn

with open("merged.txt") as f:
    print(f.read())
```

OUTPUT

```
Hello from part 1
Hello from part 2
```

Concept: combining files by streaming each source into one destination file.

Section 13 · File Handling (continued)

Problem 242. CSV Write and Read

Using the csv module, write a header and three student rows to students.csv, then read the file back and print each row.

SOLUTION

```
import csv

rows = [
    ["name", "marks"],      # header row
    ["Asha", 85],
    ["Ravi", 92],
    ["Meera", 78],
]

with open("students.csv", "w", newline="") as f: #
    newline="" avoids blank rows
    csv.writer(f).writerows(rows)

with open("students.csv", newline="") as f:
    for row in csv.reader(f):    # each row comes back as
        a list of strings
        print(row)
```

OUTPUT

```
['name', 'marks']
['Asha', '85']
['Ravi', '92']
['Meera', '78']
```

Concept: csv.writer/csv.reader for tabular data; CSV always reads back as strings.

Section 13 · File Handling (continued)

Problem 243. JSON Dump and Load

Save a dictionary describing a student to student.json with `json.dump()`, then load it back with `json.load()` and access its fields. Show that types survive the round trip.

SOLUTION

```
import json

student = {"name": "Asha", "marks": [85, 90, 78], "passed": True}

with open("student.json", "w") as f:
    json.dump(student, f, indent=2)  # indent makes the file human-readable

with open("student.json") as f:
    loaded = json.load(f)           # JSON types map back to Python types

print(loaded)
print("Name:", loaded["name"])
print("Marks are still a list:", type(loaded["marks"]) is list)
```

OUTPUT

```
{'name': 'Asha', 'marks': [85, 90, 78], 'passed': True}
Name: Asha
Marks are still a list: True
```

Concept: `json.dump()/json.load()` for structured data that preserves lists, booleans and numbers.

Exception Handling & Standard Modules

(Problems 244–258)

Problem 244. Safe Division with try/except

Write a function that divides two numbers. If the divisor is zero, catch the error and return a friendly message instead of crashing.

SOLUTION

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError:  
        return "Cannot divide by zero!"  
  
print(safe_divide(10, 2))  
print(safe_divide(5, 0))
```

OUTPUT

```
5.0  
Cannot divide by zero!
```

Concept: try/except catching a specific exception (ZeroDivisionError).

Problem 245. Multiple except Blocks

Loop over a list containing a numeric string, a non-numeric string, and None. Try to convert each to an integer and handle ValueError and TypeError with separate messages.

SOLUTION

```
values = ["10", "abc", None]

for v in values:
    try:
        print(int(v) * 2)
    except ValueError:
        print(f"ValueError: cannot convert {v!r}")
    except TypeError:
        print(f"TypeError: got a {type(v).__name__}")
```

OUTPUT

```
20
ValueError: cannot convert 'abc'
TypeError: got a NoneType
```

Concept: multiple except blocks — each catches a different exception type.

Problem 246. else and finally Clauses

Write a function that converts text to an integer. Use else to print the result only when conversion succeeds, and finally to print a cleanup message that always runs.

SOLUTION

```
def read_number(text):
    try:
        n = int(text)
    except ValueError:
        print("Conversion failed for", repr(text))
    else:
        # runs only if NO exception
        # happened
        print("Converted:", n)
    finally:
        # always runs, success or
        # failure
        print("-- cleanup done --")

read_number("42")
read_number("hello")
```

OUTPUT

```
Converted: 42
-- cleanup done --
Conversion failed for 'hello'
-- cleanup done --
```

Concept: try/except/else/finally — full exception-handling structure.

Problem 247. Raise an Exception

Write a function `set_age` that raises a `ValueError` when given a negative age. Call it with a valid and an invalid value, catching the error for the invalid one.

SOLUTION

```
def set_age(age):  
    if age < 0:  
        raise ValueError("Age cannot be negative")  
    print("Age set to", age)  
  
set_age(25)  
try:  
    set_age(-3)  
except ValueError as e:  
    print("Caught:", e)
```

OUTPUT

```
Age set to 25  
Caught: Age cannot be negative
```

Concept: `raise` statement to signal errors, and catching the exception object with `as`.

Problem 248. Custom Exception Class

Create a custom exception `InsufficientFundsError`. Write a `withdraw` function that raises it when the amount exceeds the balance, and demonstrate both a successful and a failing withdrawal.

SOLUTION

```
class InsufficientFundsError(Exception):
    """Raised when a withdrawal exceeds the balance."""

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(f"Need {amount}, have {balance}")
    return balance - amount

print("New balance:", withdraw(100, 30))
try:
    withdraw(100, 500)
except InsufficientFundsError as e:
    print("Error:", e)
```

OUTPUT

```
New balance: 70
Error: Need 500, have 100
```

Concept: defining a custom exception by inheriting from `Exception`.

Problem 249. Validate Integer Input with a Retry Loop

Keep asking the user for their age until they type a valid integer. Use try/except inside a while loop.

SOLUTION

```
while True:
    text = input("Enter your age: ")
    try:
        age = int(text)
        break          # valid input, leave the
loop
    except ValueError:
        print("Not a number, try again.")

print("Age accepted:", age)
```

OUTPUT

```
Enter your age: abc
Not a number, try again.
Enter your age: twenty
Not a number, try again.
Enter your age: 25
Age accepted: 25
```

Concept: input validation — while True + try/except + break retry pattern.

Problem 250. Current Date and Time Formatting

Print the current date and time in three formats: ISO-style date, a long readable date with time, and just the weekday name.

SOLUTION

```
from datetime import datetime

now = datetime.now()
print(now.strftime("%Y-%m-%d"))
print(now.strftime("%d %B %Y, %I:%M %p"))
print(now.strftime("%A"))
```

OUTPUT

```
2026-07-02
02 July 2026, 10:30 AM
Thursday
```

Concept: `datetime.now()` and `strftime` format codes (`%Y`, `%B`, `%I:%M %p`, `%A`).

Problem 251. Age Calculator from Birthdate

Given a birthdate, compute the person's age in whole years, correctly handling whether the birthday has occurred yet this year.

SOLUTION

```
from datetime import date

def calculate_age(birth):
    today = date(2026, 7, 2)    # fixed date for a
    repeatable example
    age = today.year - birth.year
    # subtract 1 if the birthday has not happened yet this
    year
    if (today.month, today.day) < (birth.month,
    birth.day):
        age -= 1
    return age

print(calculate_age(date(2000, 9, 15)))
print(calculate_age(date(2000, 5, 1)))
```

OUTPUT

```
25
26
```

Concept: date arithmetic with year difference and tuple comparison for the birthday check.

Problem 252. Days Between Two Dates

Compute how many days lie between January 1, 2026 and July 2, 2026.

SOLUTION

```
from datetime import date

d1 = date(2026, 1, 1)
d2 = date(2026, 7, 2)
diff = d2 - d1           # subtracting dates gives a
timedelta
print(diff.days, "days")
```

OUTPUT

```
182 days
```

Concept: date subtraction produces a timedelta with a .days attribute.

Problem 253. Add Days to a Date

Starting from July 2, 2026, print the date 30 days later and the date 7 days earlier.

SOLUTION

```
from datetime import date, timedelta

start = date(2026, 7, 2)
print("30 days later:", start + timedelta(days=30))
print("7 days earlier:", start - timedelta(days=7))
```

OUTPUT

```
30 days later: 2026-08-01
7 days earlier: 2026-06-25
```

Concept: timedelta for date arithmetic (adding and subtracting days).

Section 14 · Exception Handling & Standard Modules (continued)

Problem 254. Dice Roll Simulator

Simulate rolling a six-sided die five times and print the results on one line.

SOLUTION

```
import random

for _ in range(5):
    print(random.randint(1, 6), end=" ")    # 1 and 6 are
both included
print()
```

OUTPUT

```
3 6 2 5 1
(values vary on each run)
```

Concept: `random.randint` for uniform random integers in an inclusive range.

Problem 255. OTP Generator

Generate a 6-digit one-time password (OTP) made of random digits.

SOLUTION

```
import random

otp = "".join(random.choice("0123456789") for _ in
range(6))
print("Your OTP is:", otp)
```

OUTPUT

```
Your OTP is: 483920
(digits vary on each run)
```

Concept: `random.choice` inside a generator expression, joined into a string.

Problem 256. Random Password Generator

Generate a 12-character password using letters, digits, and a few special symbols.

SOLUTION

```
import random
import string

chars = string.ascii_letters + string.digits + "!@#$$%"
password = "".join(random.choice(chars) for _ in
range(12))
print("Password:", password)
```

OUTPUT

```
Password: k7R!pQ2z@Wf9
(characters vary on each run)
```

Concept: the string module's ready-made character sets combined with random.choice.

Section 14 · Exception Handling & Standard Modules (continued)

Problem 257. Math Module Tour

Use the math module to compute a square root, ceiling, floor, pi rounded to 4 decimals, a power, and a factorial.

SOLUTION

```
import math

print(math.sqrt(49))
print(math.ceil(4.2), math.floor(4.8))
print(round(math.pi, 4))
print(math.pow(2, 10))      # always returns a float
print(math.factorial(5))
```

OUTPUT

```
7.0
5 4
3.1416
1024.0
120
```

Concept: common math module functions — sqrt, ceil, floor, pi, pow, factorial.

Problem 258. os Module Basics

Print the current working directory, list the files inside it, and check whether a file named notes.txt exists.

SOLUTION

```
import os

print("Current dir:", os.getcwd())
for name in os.listdir("."):    # "." means the current
    print(" -", name)          directory
print("notes.txt exists?", os.path.exists("notes.txt"))
```

OUTPUT

```
Current dir: /home/user/projects
- data.csv
- main.py
- notes.txt
notes.txt exists? True
```

Concept: os.getcwd, os.listdir, and os.path.exists for filesystem basics.

Searching, Sorting & DSA Classics

(Problems 259–283)

Problem 259. Linear Search

Search for a target value in a list by checking each element one by one. Return its index, or -1 if not found.

SOLUTION

```
def linear_search(items, target):  
    for i, value in enumerate(items):  
        if value == target:  
            return i  
    return -1  
  
nums = [7, 3, 9, 14, 5]  
print(linear_search(nums, 14))  
print(linear_search(nums, 100))
```

OUTPUT

```
3  
-1
```

Concept: linear search — $O(n)$ scan with enumerate.

Section 15 · Searching, Sorting & DSA Classics (continued)

Problem 260. Binary Search (Iterative and Recursive)

Implement binary search on a sorted list twice: once with a loop and once with recursion. Return the index of the target or -1.

SOLUTION

```
def binary_search_iter(a, target):
    lo, hi = 0, len(a) - 1
    while lo ≤ hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1      # target is in the right half
        else:
            hi = mid - 1     # target is in the left half
    return -1

def binary_search_rec(a, target, lo=0, hi=None):
    if hi is None:
        hi = len(a) - 1
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if a[mid] == target:
        return mid
    if a[mid] < target:
        return binary_search_rec(a, target, mid + 1, hi)
    return binary_search_rec(a, target, lo, mid - 1)

nums = [2, 5, 8, 12, 16, 23, 38]
print(binary_search_iter(nums, 23))
print(binary_search_rec(nums, 5))
print(binary_search_iter(nums, 99))
```

OUTPUT

```
5
1
-1
```

Concept: binary search — $O(\log n)$ halving on sorted data, iterative vs recursive.

Problem 261. Bubble Sort

Sort a list with bubble sort: repeatedly swap adjacent out-of-order elements. Stop early if a full pass makes no swaps.

SOLUTION

```
def bubble_sort(a):
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):      # last i items are
already in place
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:                # already sorted,
stop early
            break
    return a

print(bubble_sort([64, 25, 12, 22, 11]))
```

OUTPUT

```
[11, 12, 22, 25, 64]
```

Concept: bubble sort with the early-exit "swapped" optimization.

Problem 262. Selection Sort

Sort a list with selection sort: find the minimum of the unsorted part and swap it into position.

SOLUTION

```
def selection_sort(a):
    for i in range(len(a) - 1):
        min_idx = i
        for j in range(i + 1, len(a)):
            if a[j] < a[min_idx]:
                min_idx = j          # remember the
smallest so far
        a[i], a[min_idx] = a[min_idx], a[i]
    return a

print(selection_sort([64, 25, 12, 22, 11]))
```

OUTPUT

```
[11, 12, 22, 25, 64]
```

Concept: selection sort — select the minimum, swap into the sorted prefix.

Problem 263. Insertion Sort

Sort a list with insertion sort: take each element and shift it left into its correct place, like sorting playing cards in hand.

SOLUTION

```
def insertion_sort(a):
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j ≥ 0 and a[j] > key:
            a[j + 1] = a[j]          # shift bigger
items right
            j -= 1
        a[j + 1] = key              # drop key into its
slot
    return a

print(insertion_sort([64, 25, 12, 22, 11]))
```

OUTPUT

```
[11, 12, 22, 25, 64]
```

Concept: insertion sort — build a sorted prefix by shifting and inserting.

Problem 264. Merge Sort

Sort a list with merge sort: split it in half, sort each half recursively, then merge the two sorted halves.

SOLUTION

```
def merge_sort(a):
    if len(a) ≤ 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    return merged + left[i:] + right[j:]    # add whatever
is left over

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

OUTPUT

```
[3, 9, 10, 27, 38, 43, 82]
```

Concept: merge sort — divide and conquer with an $O(n)$ merge step, $O(n \log n)$ overall.

Problem 265. Quick Sort

Sort a list with quick sort: pick a pivot, partition into smaller/equal/larger groups, and recurse on the outer groups.

SOLUTION

```
def quick_sort(a):
    if len(a) ≤ 1:
        return a
    pivot = a[len(a) // 2]
    smaller = [x for x in a if x < pivot]
    equal = [x for x in a if x == pivot]
    larger = [x for x in a if x > pivot]
    return quick_sort(smaller) + equal +
    quick_sort(larger)

print(quick_sort([33, 10, 55, 71, 29, 3, 18]))
```

OUTPUT

```
[3, 10, 18, 29, 33, 55, 71]
```

Concept: quick sort — partition around a pivot using list comprehensions.

Problem 266. Counting Sort (Frequency Idea)

Sort a list of small non-negative integers without comparisons: count how many times each value appears, then rebuild the list in order.

SOLUTION

```
def counting_sort(nums):
    counts = [0] * (max(nums) + 1)  # one slot per
    possible value
    for n in nums:
        counts[n] += 1
    result = []
    for value, c in enumerate(counts):
        result.extend([value] * c)  # emit each value c
    times
    return result

print(counting_sort([4, 2, 2, 8, 3, 3, 1]))
```

OUTPUT

```
[1, 2, 2, 3, 3, 4, 8]
```

Concept: counting sort — frequency array instead of comparisons, $O(n + k)$.

Problem 267. Stack Using a List

Use a plain list as a stack: push three items, pop one, and peek at the new top.

SOLUTION

```
stack = []
for item in [10, 20, 30]:
    stack.append(item)           # push onto the top
    print("Pushed", item, "→", stack)

print("Popped:", stack.pop())    # pop removes the top
                                # (LIFO)
print("Top is now:", stack[-1]) # peek without
                                # removing
print("Stack:", stack)
```

OUTPUT

```
Pushed 10 → [10]
Pushed 20 → [10, 20]
Pushed 30 → [10, 20, 30]
Popped: 30
Top is now: 20
Stack: [10, 20]
```

Concept: stack (LIFO) via list append/pop, both $O(1)$ at the end.

Problem 268. Queue Using collections.deque

Model a waiting line with a deque: enqueue three people at the back and serve two from the front.

SOLUTION

```
from collections import deque

queue = deque()
for person in ["Asha", "Bala", "Chitra"]:
    queue.append(person)          # enqueue at the back

print("Queue:", list(queue))
print("Served:", queue.popleft()) # dequeue from the
front, O(1)
print("Served:", queue.popleft())
print("Waiting:", list(queue))
```

OUTPUT

```
Queue: ['Asha', 'Bala', 'Chitra']
Served: Asha
Served: Bala
Waiting: ['Chitra']
```

Concept: queue (FIFO) — deque gives O(1) popleft, unlike list.pop(0).

Problem 269. Balanced Parentheses Checker

Check whether a string of brackets () [] { } is balanced: every opener has a matching closer in the correct order.

SOLUTION

```
def is_balanced(s):
    pairs = {"(": ")", "[": "]", "{": "}"
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False        # wrong closer or
nothing to close
            return not stack        # leftovers mean
unclosed openers

print(is_balanced("{[()()]}"))
print(is_balanced("([)]"))
print(is_balanced("(("))
```

OUTPUT

```
True
False
False
```

Concept: stack-based matching — the classic use case for LIFO.

Problem 270. Two Sum with a Hash Map

Given a list and a target, return the indices of two numbers that add up to the target, in a single pass using a dictionary.

SOLUTION

```
def two_sum(nums, target):  
    seen = {}                                # value → index  
    for i, n in enumerate(nums):  
        if target - n in seen:               # have we seen the  
partner?  
            return [seen[target - n], i]  
        seen[n] = i  
    return []  
  
print(two_sum([2, 7, 11, 15], 9))  
print(two_sum([3, 2, 4], 6))
```

OUTPUT

```
[0, 1]  
[1, 2]
```

Concept: hash map lookup turns an $O(n^2)$ pair search into $O(n)$.

Problem 271. Majority Element

Find the element that appears more than $n/2$ times using the Boyer–Moore voting algorithm in $O(n)$ time and $O(1)$ space.

SOLUTION

```
def majority_element(nums):
    candidate, count = None, 0
    for n in nums:
        if count == 0:
            candidate = n          # adopt a new
candidate
            count += 1 if n == candidate else -1
    return candidate

print(majority_element([2, 2, 1, 1, 2, 2, 3]))
```

OUTPUT

2

Concept: Boyer–Moore voting — pairing off different elements cancels them out.

Problem 272. Kadane's Maximum Subarray

Find the largest sum of any contiguous subarray using Kadane's algorithm.

SOLUTION

```
def max_subarray(nums):
    best = current = nums[0]
    for n in nums[1:]:
        current = max(n, current + n)    # extend the run
        # or restart at n
        best = max(best, current)
    return best

print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
```

OUTPUT

6

Concept: Kadane's algorithm — dynamic programming with a running best, $O(n)$.

Problem 273. Matrix Transpose

Transpose a 2×3 matrix (turn rows into columns) using zip.

SOLUTION

```
matrix = [[1, 2, 3],
          [4, 5, 6]]

transposed = [list(row) for row in zip(*matrix)] # *
unpacks the rows
for row in transposed:
    print(row)
```

OUTPUT

```
[1, 4]
[2, 5]
[3, 6]
```

Concept: `zip(*matrix)` — argument unpacking to regroup rows as columns.

Problem 274. Matrix Multiplication

Multiply two 2×2 matrices using the classic triple nested loop.

SOLUTION

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

rows, cols, inner = len(A), len(B[0]), len(B)
C = [[0] * cols for _ in range(rows)]
for i in range(rows):
    for j in range(cols):
        for k in range(inner):
            C[i][j] += A[i][k] * B[k][j]    # row of A dot
column of B

for row in C:
    print(row)
```

OUTPUT

```
[19, 22]
[43, 50]
```

Concept: matrix multiplication — dot product of row and column, $O(n^3)$.

Problem 275. Rotate a Matrix 90 Degrees

Rotate a 3×3 matrix 90 degrees clockwise by reversing the rows and then transposing.

SOLUTION

```
def rotate_90(matrix):  
    # reverse the row order, then transpose with zip  
    return [list(row) for row in zip(*matrix[::-1])]  
  
m = [[1, 2, 3],  
      [4, 5, 6],  
      [7, 8, 9]]  
for row in rotate_90(m):  
    print(row)
```

OUTPUT

```
[7, 4, 1]  
[8, 5, 2]  
[9, 6, 3]
```

Concept: 90° clockwise rotation = reverse rows + transpose.

Section 15 · Searching, Sorting & DSA Classics (continued)

Problem 276. Singly Linked List: Create, Insert, Traverse

Build a singly linked list class with insert-at-end and traversal methods, then insert three values and print the list.

SOLUTION

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_end(self, data):
        node = Node(data)
        if not self.head:           # empty list: node becomes head
            self.head = node
            return
        cur = self.head
        while cur.next:             # walk to the last node
            cur = cur.next
        cur.next = node

    def traverse(self):
        cur = self.head
        while cur:
            print(cur.data, end=" → ")
            cur = cur.next
        print("None")

ll = LinkedList()
for x in [10, 20, 30]:
    ll.insert_end(x)
ll.traverse()
```

OUTPUT

```
10 → 20 → 30 → None
```

Concept: singly linked list — nodes chained by `.next` references.

Section 15 · Searching, Sorting & DSA Classics (continued)

Problem 277. Reverse a Linked List

Reverse a singly linked list in place by flipping each node's next pointer, using three pointers: prev, cur, and nxt.

SOLUTION

```
class Node:
    def __init__(self, data, next=None):
        self.data, self.next = data, next

def build(values):
    # helper: list → linked list
    head = None
    for v in reversed(values):
        head = Node(v, head)
    return head

def show(head):
    out = []
    while head:
        out.append(str(head.data))
        head = head.next
    print(" → ".join(out))

def reverse(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next          # save the rest of the list
        cur.next = prev         # flip the pointer backwards
        prev = cur
        cur = nxt
    return prev                # prev is the new head

head = build([1, 2, 3, 4, 5])
show(head)
show(reverse(head))
```

OUTPUT

```
1 → 2 → 3 → 4 → 5
5 → 4 → 3 → 2 → 1
```

Concept: in-place linked list reversal with the three-pointer technique.

Section 15 · Searching, Sorting & DSA Classics (continued)

Problem 278. Find the Middle of a Linked List

Find the middle node of a linked list in one pass using slow and fast pointers (fast moves two steps per slow step).

SOLUTION

```
class Node:
    def __init__(self, data, next=None):
        self.data, self.next = data, next

def build(values):
    head = None
    for v in reversed(values):
        head = Node(v, head)
    return head

def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next           # 1 step
        fast = fast.next.next      # 2 steps
    return slow.data               # when fast finishes, slow is halfway

print(middle(build([1, 2, 3, 4, 5])))
print(middle(build([1, 2, 3, 4, 5, 6])))
```

OUTPUT

```
3
4
```

Concept: slow/fast (tortoise and hare) two-pointer technique.

Section 15 · Searching, Sorting & DSA Classics (continued)

Problem 279. Remove Duplicates from a Sorted Linked List

Given a sorted linked list, delete duplicate nodes so each value appears once.

SOLUTION

```
class Node:
    def __init__(self, data, next=None):
        self.data, self.next = data, next

def build(values):
    head = None
    for v in reversed(values):
        head = Node(v, head)
    return head

def show(head):
    out = []
    while head:
        out.append(str(head.data))
        head = head.next
    print(" → ".join(out))

def remove_duplicates(head):
    cur = head
    while cur and cur.next:
        if cur.data == cur.next.data:
            cur.next = cur.next.next    # unlink the duplicate node
        else:
            cur = cur.next
    return head

head = build([1, 1, 2, 3, 3, 3, 4])
show(remove_duplicates(head))
```

OUTPUT

```
1 → 2 → 3 → 4
```

Concept: node deletion by re-linking — duplicates in sorted data are always adjacent.

Problem 280. Missing Number from 1..n

A list contains all numbers from 1 to n except one. Find the missing number using the sum formula, without sorting.

SOLUTION

```
def missing_number(nums, n):  
    expected = n * (n + 1) // 2      # sum of 1..n (Gauss  
    formula)  
    return expected - sum(nums)  
  
print(missing_number([1, 2, 4, 5, 6], 6))  
print(missing_number([2, 3, 1, 5], 5))
```

OUTPUT

```
3  
4
```

Concept: arithmetic-series sum formula — expected minus actual reveals the gap.

Problem 281. First Repeating Element

Find the leftmost element in a list that appears more than once. Return None if all elements are unique.

SOLUTION

```
from collections import Counter

def first_repeating(nums):
    counts = Counter(nums)           # count everything
    first
    for n in nums:                   # then scan left to
    right
        if counts[n] > 1:
            return n
    return None

print(first_repeating([10, 5, 3, 4, 3, 5, 6]))
print(first_repeating([1, 2, 3]))
```

OUTPUT

```
5
None
```

Concept: two-pass Counter approach — count, then find the first with count > 1.

Section 15 · Searching, Sorting & DSA Classics (continued)

Problem 282. Search in a Rotated Sorted Array

A sorted array has been rotated at an unknown pivot (e.g. [4,5,6,7,0,1,2]). Find a target in $O(\log n)$ using a modified binary search.

SOLUTION

```
def search_rotated(a, target):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[lo] <= a[mid]:           # left half is sorted
            if a[lo] <= target < a[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                       # right half is sorted
            if a[mid] < target <= a[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1

print(search_rotated([4, 5, 6, 7, 0, 1, 2], 0))
print(search_rotated([4, 5, 6, 7, 0, 1, 2], 3))
```

OUTPUT

```
4
-1
```

Concept: modified binary search — one half is always sorted; decide which half to keep.

Problem 283. Merge Overlapping Intervals

Given a list of [start, end] intervals, merge all overlapping intervals and return the result.

SOLUTION

```
def merge_intervals(intervals):
    intervals.sort()                # sort by start
    time
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start ≤ merged[-1][1]:    # overlaps the
previous interval
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged

print(merge_intervals([[1, 3], [2, 6], [8, 10], [15,
18]]))
print(merge_intervals([[5, 7], [1, 4], [4, 6]]))
```

OUTPUT

```
[[1, 6], [8, 10], [15, 18]]
[[1, 7]]
```

Concept: sort by start, then greedy merge — the standard intervals pattern.

SECTION 16

Real-World Mini Programs & Interview Classics

(Problems 284–300)

Problem 284. FizzBuzz

Print numbers 1 to 15, but print "Fizz" for multiples of 3, "Buzz" for multiples of 5, and "FizzBuzz" for multiples of both.

SOLUTION

```
for n in range(1, 16):
    if n % 15 == 0:           # divisible by both 3 and 5
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

OUTPUT

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Concept: modulo checks with elif ordering — test the combined case first.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 285. Password Strength Checker

Score a password one point each for: length 8+, a lowercase letter, an uppercase letter, a digit, and a special character. Report Weak, Medium, Good, or Strong.

SOLUTION

```
def strength(pw):
    score = 0
    if len(pw) ≥ 8:
        score += 1
    if any(c.islower() for c in pw):
        score += 1
    if any(c.isupper() for c in pw):
        score += 1
    if any(c.isdigit() for c in pw):
        score += 1
    if any(c in "!@#$%^&*" for c in pw):
        score += 1
    levels = {5: "Strong", 4: "Good", 3: "Medium"}
    return levels.get(score, "Weak")    # anything below 3 is Weak

for pw in ["abc", "abcdef12", "Abcdef12", "Abcdef12!"]:
    print(pw, "→", strength(pw))
```

OUTPUT

```
abc → Weak
abcdef12 → Medium
Abcdef12 → Good
Abcdef12! → Strong
```

Concept: any() with generator expressions for rule checks, dict.get with a default.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 286. Number Guessing Game

The computer picks a random number from 1 to 100. The player keeps guessing; after each guess print "Too low!" or "Too high!" until they get it right.

SOLUTION

```
import random

secret = random.randint(1, 100)
attempts = 0
while True:
    guess = int(input("Guess (1-100): "))
    attempts += 1
    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
    else:
        print(f"Correct! You got it in {attempts} attempts.")
        break
```

OUTPUT

```
Guess (1-100): 50
Too low!
Guess (1-100): 75
Too high!
Guess (1-100): 62
Too low!
Guess (1-100): 68
Correct! You got it in 4 attempts.
```

Concept: game loop with while True, break, and comparison-based feedback.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 287. Rock-Paper-Scissors

Play one round of rock-paper-scissors against the computer. Use a dictionary that maps each move to the move it beats.

SOLUTION

```
import random

beats = {"rock": "scissors", "paper": "rock", "scissors":
"paper"}
player = input("rock/paper/scissors: ").lower()
computer = random.choice(list(beats))
print("Computer chose:", computer)

if player == computer:
    print("Draw!")
elif beats[player] == computer:      # player's move beats
    computer's
    print("You win!")
else:
    print("You lose!")
```

OUTPUT

```
rock/paper/scissors: rock
Computer chose: scissors
You win!
```

Concept: encoding game rules in a dictionary instead of many if statements.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 288. Simple To-Do CLI

Build a tiny command-line to-do app that understands: add <task>, list, remove <number>, and quit.

SOLUTION

```
todos = []
while True:
    cmd = input("> ").split(maxsplit=1) # ["add", "buy milk"]
    if cmd[0] == "add":
        todos.append(cmd[1])
        print("Added:", cmd[1])
    elif cmd[0] == "list":
        for i, task in enumerate(todos, 1):
            print(f"{i}. {task}")
    elif cmd[0] == "remove":
        removed = todos.pop(int(cmd[1]) - 1)
        print("Removed:", removed)
    elif cmd[0] == "quit":
        print("Bye!")
        break
    else:
        print("Commands: add, list, remove, quit")
```

OUTPUT

```
> add buy milk
Added: buy milk
> add pay bills
Added: pay bills
> list
1. buy milk
2. pay bills
> remove 1
Removed: buy milk
> list
1. pay bills
> quit
Bye!
```

Concept: command-dispatch loop with `split(maxsplit=1)` and list mutation.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 289. ATM Simulation

Simulate an ATM with a menu loop: check balance, deposit, withdraw (rejecting overdrafts), and exit.

SOLUTION

```
balance = 1000.0
while True:
    print("1) Balance  2) Deposit  3) Withdraw  4) Exit")
    choice = input("Choose: ")
    if choice == "1":
        print(f"Balance: Rs.{balance:.2f}")
    elif choice == "2":
        balance += float(input("Amount: "))
        print("Deposited.")
    elif choice == "3":
        amount = float(input("Amount: "))
        if amount > balance:
            print("Insufficient funds!")
        else:
            balance -= amount
            print("Withdrawn.")
    elif choice == "4":
        print("Goodbye!")
        break
```

OUTPUT

```
1) Balance  2) Deposit  3) Withdraw  4) Exit
Choose: 2
Amount: 500
Deposited.
1) Balance  2) Deposit  3) Withdraw  4) Exit
Choose: 3
Amount: 2000
Insufficient funds!
1) Balance  2) Deposit  3) Withdraw  4) Exit
Choose: 1
Balance: Rs.1500.00
1) Balance  2) Deposit  3) Withdraw  4) Exit
Choose: 4
Goodbye!
```

Concept: menu-driven state machine — a loop mutating one balance variable.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 290. Tic-Tac-Toe (Compact)

Implement a compact two-player tic-tac-toe on a 3×3 board. Players type positions 1–9; detect wins and draws.

SOLUTION

```
board = [" "] * 9
WINS = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]

def show():
    for i in (0, 3, 6):
        print("|".join(board[i:i+3]))

def wins(mark):
    return any(all(board[i] == mark for i in line) for line in WINS)

player = "X"
for turn in range(9):
    pos = int(input(f"{player} moves (1-9): ")) - 1
    while board[pos] != " ":
        # re-ask if the cell is taken
        pos = int(input("Taken, pick again: ")) - 1
    board[pos] = player
    show()
    if wins(player):
        print(player, "wins!")
        break
    player = "O" if player == "X" else "X"
else:
    # loop finished with no winner
    print("Draw!")
```

OUTPUT

```
X moves (1-9): 1
X| |
| |
| |
O moves (1-9): 5
X| |
|O|
| |
X moves (1-9): 2
X|X|
|O|
| |
O moves (1-9): 9
X|X|
|O|
| |O
X moves (1-9): 3
X|X|X
|O|
| |O
X wins!
```

Concept: board as a flat list, win lines as index tuples, for/else for the draw case.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 291. Email Validator with Regex

Validate email addresses with a regular expression: a local part, an @, a domain, and a top-level domain of at least two letters.

SOLUTION

```
import re

PATTERN = r"^[\\w.+\\-]+@[\\w.-]+\\. [a-zA-Z]{2,}$"

def is_valid_email(email):
    return re.match(PATTERN, email) is not None

tests = ["user@example.com", "a.b+tag@mail.co.in",
         "bad@@mail.com", "no-at-sign.com"]
for t in tests:
    print(t, "→", is_valid_email(t))
```

OUTPUT

```
user@example.com → True
a.b+tag@mail.co.in → True
bad@@mail.com → False
no-at-sign.com → False
```

Concept: `re.match` with anchors (^ and \$) and character classes for validation.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 292. Extract Numbers from Text with Regex

Pull every number (integers and decimals) out of a sentence and convert each to int or float.

SOLUTION

```
import re

text = "Order #66: 3 items cost Rs. 1499.50, delivered in 2 days"
numbers = re.findall(r"\d+(?:\.\d+)?", text)    # optional decimal part
print(numbers)

values = [float(n) if "." in n else int(n) for n in numbers]
print(values)
```

OUTPUT

```
['66', '3', '1499.50', '2']
[66, 3, 1499.5, 2]
```

Concept: `re.findall` with a non-capturing group `(?:...)` for optional decimals.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 293. Phone Number Formatter

Strip all non-digits from a raw phone string and format a 10-digit number as (XXX) XXX-XXXX; report anything else as invalid.

SOLUTION

```
def format_phone(raw):
    digits = "".join(ch for ch in raw if ch.isdigit())
    if len(digits) == 10:
        return f"({digits[:3]}) {digits[3:6]}-{digits[6:]}"
    return "Invalid number"

print(format_phone("9876543210"))
print(format_phone("98-7654 3210"))
print(format_phone("12345"))
```

OUTPUT

```
(987) 654-3210
(987) 654-3210
Invalid number
```

Concept: cleaning input with `str.isdigit`, then formatting with slices and f-strings.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 294. Unit Converter (Length and Weight)

Convert between km/miles and kg/lb using a dictionary of conversion factors.

SOLUTION

```
def convert(value, unit):
    factors = {"km": ("miles", 0.621371), "miles": ("km",
1.60934),
               "kg": ("lb", 2.20462),      "lb": ("kg",
0.453592)}
    target, factor = factors[unit]
    return f"{value} {unit} = {value * factor:.2f} {target}"

print(convert(5, "km"))
print(convert(70, "kg"))
print(convert(10, "miles"))
```

OUTPUT

```
5 km = 3.11 miles
70 kg = 154.32 lb
10 miles = 16.09 km
```

Concept: lookup-table design — data (factors) separated from logic.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 295. Word-Count Report of a Paragraph

For a paragraph, report the total word count, the number of unique words, and the three most common words.

SOLUTION

```
from collections import Counter

paragraph = ("Python is fun. Python is easy to learn "
            "and easy to read.")
words = paragraph.lower().replace(".", "").split()
counts = Counter(words)

print("Total words:", len(words))
print("Unique words:", len(counts))
print("Top 3:", counts.most_common(3))
```

OUTPUT

```
Total words: 12
Unique words: 8
Top 3: [('python', 2), ('is', 2), ('easy', 2)]
```

Concept: text normalization (lower/replace/split) plus
Counter.most_common.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 296. Simple Contact Book with a Dict

Build a contact book backed by a dictionary with add, search, and delete functions, and demonstrate each.

SOLUTION

```
contacts = {}

def add(name, phone):
    contacts[name] = phone
    print("Added", name)

def search(name):
    print(name, ":", contacts.get(name, "Not found"))

def delete(name):
    if contacts.pop(name, None):      # pop with default avoids KeyError
        print("Deleted", name)
    else:
        print(name, "not found")

add("Asha", "98765 43210")
add("Bala", "91234 56780")
search("Asha")
delete("Bala")
search("Bala")
```

OUTPUT

```
Added Asha
Added Bala
Asha : 98765 43210
Deleted Bala
Bala : Not found
```

Concept: dict as a key-value store — get and pop with defaults for safe access.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 297. Countdown Timer Logic

Given a number of seconds, print a countdown in MM:SS format down to 00:00 (pure logic, no sleeping required).

SOLUTION

```
def countdown(total_seconds):
    for remaining in range(total_seconds, -1, -1):
        mins, secs = divmod(remaining, 60)    # quotient
        and remainder
        print(f"{mins:02d}:{secs:02d}")
        # time.sleep(1) would go here for a real timer

countdown(63)
```

OUTPUT

```
01:03
01:02
01:01
01:00
00:59
(...and so on, one line per second, down to 00:00)
```

Concept: divmod for minutes/seconds and :02d zero-padded formatting.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 298. Roman Numeral Converter (Both Ways)

Convert an integer to a Roman numeral and a Roman numeral back to an integer, handling subtractive forms like IV and CM.

SOLUTION

```
VALUES = [(1000, "M"), (900, "CM"), (500, "D"), (400, "CD"),
          (100, "C"), (90, "XC"), (50, "L"), (40, "XL"),
          (10, "X"), (9, "IX"), (5, "V"), (4, "IV"), (1, "I")]

def to_roman(n):
    out = ""
    for value, symbol in VALUES:
        while n ≥ value:           # greedily take the biggest symbol
            out += symbol
            n -= value
    return out

def from_roman(s):
    m = {"I": 1, "V": 5, "X": 10, "L": 50,
         "C": 100, "D": 500, "M": 1000}
    total = 0
    for i, ch in enumerate(s):
        # a smaller value before a bigger one means subtract (IV = 4)
        if i + 1 < len(s) and m[ch] < m[s[i + 1]]:
            total -= m[ch]
        else:
            total += m[ch]
    return total

print(to_roman(1994))
print(from_roman("MMXXVI"))
```

OUTPUT

```
MCMXCIV
2026
```

Concept: greedy value table for encoding; look-ahead subtraction rule for decoding.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 299. Expression Evaluator for + - * /

Evaluate a space-separated arithmetic expression like "2 + 3 * 4" with correct operator precedence, without using eval.

SOLUTION

```
def evaluate(expr):
    tokens = expr.split()
    stack = [float(tokens[0])]
    i = 1
    while i < len(tokens):
        op, num = tokens[i], float(tokens[i + 1])
        if op == "*":
            stack[-1] *= num          # * and / apply immediately
        elif op == "/":
            stack[-1] /= num
        elif op == "+":
            stack.append(num)         # + and - wait on the stack
        else:
            stack.append(-num)        # "-"
        i += 2
    # summing the stack applies all the deferred + and - at once
    return sum(stack)

print(evaluate("2 + 3 * 4"))
print(evaluate("10 - 6 / 3 + 1"))
```

OUTPUT

```
14.0
9.0
```

Concept: stack-based evaluation — resolve high-precedence ops now, defer + and - via sum.

Section 16 · Real-World Mini Programs & Interview Classics (continued)

Problem 300. Temperature Statistics

From a week of temperature readings, report the minimum, maximum, average (2 decimals), and how many days were above average.

SOLUTION

```
temps = [31.4, 29.8, 33.2, 30.1, 35.6, 28.9, 32.5]

low, high = min(temps), max(temps)
avg = sum(temps) / len(temps)
hot_days = [t for t in temps if t > avg]

print(f"Min: {low}   Max: {high}")
print(f"Average: {avg:.2f}")
print(f"Days above average: {len(hot_days)}")
```

OUTPUT

```
Min: 28.9   Max: 35.6
Average: 31.64
Days above average: 3
```

Concept: built-in aggregates (min/max/sum/len) with a filtering list comprehension.